



Clustering de Protéines

PMIND

Mathilde Gauteur | M1 MIND | 21500128
Élise Thomas | M1 MIND | 21234701

Janvier 2026

Table des matières

1	Contexte et motivations	4
2	Notions clés	4
2.1	Séquences protéiques et annotation fonctionnelle	4
2.2	Graphe de similarité basé sur les alignements (BLAST/Diamond)	5
2.3	Embeddings de protéines	5
2.4	Clustering et homogénéité fonctionnelle	5
3	État de l'art	6
3.1	Clustering de protéines basé sur les graphes de similarité	6
3.2	Embeddings et deep learning pour les protéines	6
3.3	Limites des approches existantes motivant une approche hybride	6
4	Introduction au sujet	6
5	Pipeline	7
6	Données	8
6.1	Fichiers .fasta	8
6.2	Fichiers Diamond	8
7	Graphes	12
7.1	Quelques définitions	12
7.2	Illustration sur un exemple de graphe non orienté	12
8	Clustering	13
8.1	MCL (Markov Clustering Algorithm)	13
8.1.1	Algorithme	13
8.2	Louvain	14
8.2.1	Algorithme	14
8.3	Leiden	16
8.3.1	Algorithme	16
8.4	Comparaison structurelle des partitions	17
8.4.1	L'Index de Rand (RI)	17
8.4.2	Indice de Rand Ajusté (ARI)	18
9	Métriques d'évaluation de clustering	18
9.1	Exemples de partitions	19
9.1.1	Partition idéale - forte homogénéité et séparabilité	19
9.1.2	Mauvaise partition - hétérogénéité et faible séparabilité	19
9.2	Mesures d'homogénéité sur une partition P	19
9.2.1	Co-distance de P	19
9.3	Mesures de séparabilité des clusters de la partition	19
9.3.1	Coupe	19
9.3.2	Coupe Normalisée	20
9.4	Mesures basées sur les annotations	20

10 Mesures de la qualité de la prédiction d’annotation	23
10.1 Précision	23
10.2 Rappel	24
10.3 F1-score	24
11 Propagation des annotations	25
12 L’algorithme des k plus proches voisins (kNN) comme standard d’annotation fonctionnelle	26
13 Première approche	27
13.1 Étapes	27
14 Deuxième approche	27
14.1 Étapes générales	27
14.2 Combinaison des deux partitions	28
14.2.1 Fusion finale des partitions après la convergence	28
15 Protocole expérimental	30
15.1 Création des Embeddings	30
15.1.1 Choix du modèle	30
15.1.2 Descriptions de ProteinBERT et de ESM-2	30
15.1.3 Méthode de création des embeddings	31
15.1.4 Résultats de ProteinBERT	32
15.2 Graphe de similarité BLAST	33
15.2.1 Traitement du graphe BLAST	33
15.2.2 Graphe BLAST complet	34
15.2.3 Graphe BLAST réduit	35
15.3 Graphe combinant BLAST et les embeddings	36
15.3.1 Calcul de la similarité cosinus entre les embeddings	36
15.3.2 Fusion du graphe BLAST et de la similarité cosinus des embeddings	37
15.3.3 Implémentation des métriques d’évaluation	38
15.4 Clustering avec Leiden et MCL	38
15.4.1 Méthode	38
15.4.2 Résultats	40
15.5 Évaluation des partitions et de la propagation à partir des annotations PFAM	47
15.5.1 Évaluation de l’exploitabilité des clusters	47
15.5.2 Résultats de propagation pour la première approche	50
15.5.3 Résultats de propagation pour la deuxième approche	51
15.5.4 Comparaison des deux approches	51
16 Conclusion et Perspectives	53
16.1 Conclusion	53
16.2 Ouverture et Perspectives	53

1 Contexte et motivations

Les progrès majeurs réalisés ces dernières années dans les technologies de séquençage ont conduit à une augmentation massive du nombre de séquences protéiques disponibles.

Des projets de grande ampleur, notamment en métagénomique et en écologie microbienne, produisent aujourd’hui des millions de nouvelles séquences issues d’organismes cultivés ou non cultivés.

Cependant, cette explosion des données contraste fortement avec la lenteur et le coût des méthodes expérimentales permettant de déterminer la fonction biologique des protéines.

Ainsi, pour une large majorité des protéines connues, aucune annotation fonctionnelle fiable n’est disponible. On estime que moins de 25./ des protéines disposent d’une annotation validée expérimentalement, ce qui limite fortement l’exploitation biologique de ces données. L’annotation automatique apparaît donc comme une étape indispensable pour tirer parti de ces grandes bases de données protéiques.

Une approche classique pour l’annotation fonctionnelle repose sur le principe d’homologie : deux protéines ayant des séquences similaires sont supposées partager des fonctions similaires. Des outils d’alignement tels que BLAST ou Diamond sont largement utilisés pour comparer les séquences et construire des graphes de similarité protéine–protéine. Ces graphes peuvent ensuite être exploités par des algorithmes de clustering afin de regrouper les protéines en familles fonctionnelles.

Cependant, cette approche présente plusieurs limites. En particulier, les méthodes de clustering appliquées uniquement à des graphes de similarité basés sur l’alignement de séquences produisent souvent des clusters hétérogènes du point de vue fonctionnel. Il n’est pas rare d’observer, au sein d’un même cluster, des protéines associées à des annotations fonctionnelles incompatibles. Ces erreurs compromettent la qualité de l’annotation par propagation et limitent la fiabilité biologique des résultats.

Parallèlement, des avancées récentes en apprentissage profond ont conduit au développement de modèles de type « language models » appliqués aux séquences protéiques. Ces modèles permettent de représenter chaque protéine par un vecteur dense, appelé embedding, qui capture des propriétés complexes liées à la structure, à l’évolution et à la fonction des protéines. Contrairement aux alignements de séquence, ces représentations ne se limitent pas à des similarités locales et peuvent mettre en évidence des relations fonctionnelles plus subtiles.

L’objectif de ce projet est d’explorer l’intégration de ces embeddings protéiques dans le processus de clustering, en complément des graphes de similarité issus de BLAST. L’hypothèse sous-jacente est que la combinaison de ces deux sources d’information permettra d’obtenir des clusters plus homogènes du point de vue fonctionnel, et ainsi d’améliorer la qualité de l’annotation automatique des protéines.

2 Notions clés

2.1 Séquences protéiques et annotation fonctionnelle

- Une protéine est une chaîne d’acides aminés. Sa séquence détermine sa structure et sa fonction.
- Les annotations fonctionnelles décrivent l’activité ou le rôle biologique d’une protéine et proviennent de bases de données comme Pfam, InterPro ou Gene Ontology.

- Certaines annotations sont incomplètes, et des contraintes d’incompatibilité sont données : deux fonctions incompatibles ne doivent pas se retrouver dans le même cluster.

2.2 Graphe de similarité basé sur les alignements (BLAST/Diamond)

Les outils d’alignement de séquences, tels que BLAST ou Diamond, permettent de comparer deux protéines et de mesurer leur similarité à l’aide de différents scores (pourcentage d’identité, bitscore, e-value).

- BLAST : alignement local classique, précis mais lent sur de très grandes bases.
- Diamond : version optimisée pour les millions de séquences, produisant les mêmes scores.

À partir de ces comparaisons, il est possible de construire un graphe de similarité :

- chaque sommet représente une protéine
- une arête relie deux protéines si un alignement significatif existe entre elles
- le poids de l’arête correspond à un score de similarité

Ce graphe constitue la base des méthodes classiques de clustering de protéines.

2.3 Embeddings de protéines

- Un embedding de protéine est une représentation vectorielle dense d’une séquence protéique, généralement obtenue à partir d’un modèle de deep learning pré-entraîné sur de grandes bases de données de séquences, par exemple ProteinBERT ici.
- Ces modèles, inspirés des modèles de langage naturel, apprennent à capturer des régularités statistiques et biologiques dans les séquences d’acides aminés. Dans ce travail, les embeddings sont calculés directement à partir des séquences FASTA, indépendamment des alignements BLAST.
- Une fois normalisés, ces vecteurs peuvent être comparés à l’aide de la similarité cosinus afin de mesurer la proximité entre protéines dans l’espace d’embedding.

2.4 Clustering et homogénéité fonctionnelle

Le clustering vise à regrouper les protéines en ensembles cohérents, appelés clusters, supposés correspondre à des familles fonctionnelles.

La qualité d’un clustering est évaluée à l’aide de critères d’homogénéité fonctionnelle, notamment :

- le taux de compatibilité des annotations au sein des clusters
- le taux de violation des contraintes d’incompatibilité
- des métriques telles que le coefficient de silhouette, qui mesure la cohérence géométrique dans l’espace (utile pour comparer l’apport des embeddings)

Notre regard Machine Learning : considérer les embeddings comme des features vectorielles, et le clustering comme une tâche non supervisée où l’on cherche à maximiser l’homogénéité fonctionnelle.

3 État de l’art

3.1 Clustering de protéines basé sur les graphes de similarité

Les méthodes de clustering de graphes sont largement utilisées en bioinformatique pour l’analyse des réseaux de similarité protéique.

Parmi elles, l’algorithme MCL (Markov Clustering Algorithm) est particulièrement populaire dans la communauté biologique. MCL simule des marches aléatoires sur le graphe et exploite les propriétés de diffusion et d’inflation pour identifier des régions densément connectées, correspondant à des groupes de protéines similaires.

MCL présente plusieurs avantages : il ne nécessite pas de fixer a priori le nombre de clusters, il est robuste au bruit et il s’adapte bien aux graphes clairsemés issus des alignements de séquences. Ces caractéristiques expliquent son intégration dans de nombreux pipelines d’annotation protéique, tels que LAGOON-MCL.

D’autres méthodes, comme Louvain ou Leiden, optimisent la modularité du graphe et sont largement utilisées dans l’analyse de réseaux complexes. Toutefois, ces approches sont moins couramment employées en bioinformatique pour le clustering de protéines, notamment en raison de résultats parfois moins cohérents du point de vue fonctionnel.

3.2 Embeddings et deep learning pour les protéines

L’utilisation de modèles de deep learning pour l’analyse des séquences protéiques s’est fortement développée ces dernières années.

Des modèles tels que **ProteinBERT**, ESM ou ProtTrans permettent de générer des embeddings, étant des vecteurs qui capturent des propriétés complexes telles que la structure, et la fonction.

- Ces représentations peuvent avoir diverses applications, notamment la prédiction de fonction, la détection de domaines, ou encore le clustering de protéines.
- On trouve toutefois des limites, ces résultats sont moins interprétables biologiquement et l’information évolutive est parfois perdue

3.3 Limites des approches existantes motivant une approche hybride

Les approches reposant exclusivement sur les alignements de séquences perdent les relations fonctionnelles lorsque la similarité de séquence est faible.

À l’inverse, les méthodes basées uniquement sur les embeddings peuvent manquer d’interprétabilité biologique et ignorer des informations évolutives importantes.

Cela motive le développement de méthodes hybrides combinant graphes de similarité et embeddings, afin de tirer parti des avantages de chaque approche et obtenir des clusters plus homogènes et annotation plus fiable.

4 Introduction au sujet

Ces dernières années, les progrès en séquençage ont été conséquents, nous permettant d’obtenir des séquences de nombreux organismes et échantillons provenant de l’environnement. Cependant, l’analyse de ces données est un travail conséquent qui ne peut pas

être effectué par l'être humain, la quantité de données étant conséquente, et la difficulté d'obtenir des données expérimentales trop importante. Les biologistes font donc appel aux informaticiens afin que des méthodes adaptées d'apprentissage automatique puissent les aider dans leur tâche.

L'objectif général de ce projet est de déterminer les groupes fonctionnels des protéines, c'est-à-dire leurs annotations, à partir de leurs séquences d'acides aminés.

Les données étudiées sont :

- **Le Graphe de similarité (BLAST)** : données d'alignement entre les paires de protéines, calculées par la méthode BLAST. Si deux protéines se ressemblent au niveau de leur séquence d'acides aminés, on suppose qu'elles ont la même fonction et que leur score de similarité sera élevé.
- **L'Embedding (vecteur dense)** : représentation mathématique issue de l'IA. Elle capture des relations complexes provenant de la structure interne d'une protéine, différentes selon le modèle pré-entraîné utilisé. Les résultats obtenus sont différents d'un alignement BLAST.
- **Annotations fonctionnelles (target)** : certaines protéines ont une ou plusieurs annotations fonctionnelles déterminées. Certaines annotations sont incompatibles et ne doivent pas coexister dans un même cluster de protéines.

Difficultés : l'information dont nous disposons sur les annotations est très partielle : 30,8% des protéines sont annotées avec des annotations PFAM. De plus, nous ne disposons pas de notion claire sur l'incompatibilité des annotations, nous ne pouvons donc les identifier qu'en nous basant sur les données que nous possédons déjà, par exemple si elles se trouvent ensemble au sein d'une même protéine.

Objectif : améliorer le processus de clustering en ajoutant l'utilisation des embeddings de protéines à celle du graphe de similarité BLAST.

Défis : les méthodes de clustering actuellement appliquées sur le graphe BLAST ne donnent pas des clusters suffisamment homogènes. Certains clusters contiennent des protéines ayant des annotations fonctionnelles incompatibles.

5 Pipeline

Pour étudier l'ajout de l'information donnée par les embeddings au graphe de similarité BLAST, nous avons érigé deux approches que nous avons expérimentées au sein de ce projet :

1. **Première approche** : combinaison linéaire pondérée des graphes de la similarité cosinus des embeddings et de la similarité BLAST. Un algorithme de clustering sera ensuite appliqué sur le graphe obtenu.
2. **Deuxième approche** : alignement des partitions obtenues indépendamment par les graphes de la similarité cosinus des embeddings et de la similarité BLAST.

Pour chaque approche, nous suivrons la pipeline suivante :

- **Application de l'approche choisie (1 ou 2)**
- **Analyse des métriques de clustering**
 - **Mesures d'homogénéité de la partition**
 - Co-distance de P
 - **Mesures de séparabilité des clusters de la partition**
 - Coupe normalisée

- **Propagation des annotations au sein de la partition** (avec une méthode particulière pour la deuxième approche)
- **Mesures de la qualité de la prédiction d’annotation**
 - F1-score
 - Rappel
 - Précision

6 Données

6.1 Fichiers .fasta

Description Les fichiers `.fasta` contiennent les séquences protéiques utilisées dans ce projet. Chaque fichier correspond à un protéome issu de la base de données EukProt, et regroupe l’ensemble des protéines d’un organisme donné.

Chaque entrée est composée :

- d’une ligne d’en-tête contenant un identifiant unique de protéine ainsi que des informations sur l’organisme ;
- d’une séquence d’acides aminés représentée sous forme de chaîne de caractères.

Exemple

```
>Abedinium_dasypus_EP00812|sequence00001 EukProt EP00812 Abedinium_dasypus_DSEL18-54
MGCHVVTILVRRSEAPAGQDSMVKPRLKPVdqYLEESALAEADVVMRQGTITLNRSLTQLALRGVGGYTYAILNFKDIEVIRDMY
```

Dans cet exemple, `sequence00001` constitue l’identifiant unique de la protéine, et la ligne suivante correspond à sa séquence d’acides aminés.

Ces séquences constituent l’entrée des modèles d’embeddings ainsi que des outils d’alignement utilisés pour construire les graphes de similarité.

6.2 Fichiers Diamond

Choix de la source d’annotations Les fichiers d’annotations disponibles contiennent plusieurs sources hétérogènes, par exemple PFAM, SMART, Gene3D, SUPERFAMILY, ProSite, TMHMM, Phobius ou MobiDBLite. Ces sources ne décrivent pas toutes le même niveau d’information biologique. Certaines correspondent à des domaines ou familles protéiques, tandis que d’autres décrivent plutôt des propriétés structurales ou physico-chimiques globales, comme la présence d’hélices transmembranaires ou de régions désordonnées.

Pour éviter de mélanger des annotations de nature différente, nous avons choisi de restreindre l’analyse aux annotations PFAM. Ce choix permet de travailler avec une source homogène décrivant des familles et domaines protéiques. Il limite donc le risque de considérer comme fonctionnellement similaires deux protéines uniquement parce qu’elles partagent une propriété très générale, par exemple une région transmembranaire ou une zone de désordre.

Les annotations PFAM sont donc utilisées comme approximation de la similarité fonctionnelle entre protéines. Cette approximation n’est pas parfaite, car deux protéines peuvent avoir une fonction proche sans partager exactement les mêmes domaines PFAM, ou inversement partager un domaine fréquent sans être strictement équivalentes fonctionnellement. Néanmoins, PFAM constitue une source suffisamment structurée et interprétable pour évaluer l’homogénéité des clusters.

Format des données Chaque ligne du fichier d’annotations associe une protéine à une annotation. Le format général est le suivant :

Protéine	Source	Identifiant	Description
sequence03277	Pfam	PF13519	von Willebrand factor type A domain

Après filtrage, seules les lignes dont la source est **Pfam** sont conservées. Chaque protéine est ensuite représentée par l’ensemble de ses identifiants PFAM.

Statistiques globales Le tableau 1 présente les statistiques générales obtenues après extraction des annotations PFAM.

Statistique	Valeur
Nombre total de protéines étudiées	7 633 132
Nombre de protéines présentes dans les fichiers d’annotations	5 769 359
Nombre de protéines possédant au moins une annotation PFAM	2 350 666
Proportion PFAM parmi les protéines présentes dans les fichiers d’annotations	40.74%
Proportion PFAM parmi toutes les protéines étudiées	30.80%
Nombre total de lignes d’annotations PFAM	2 962 935
Nombre d’annotations PFAM distinctes	9 146

TABLE 1 – Statistiques globales des annotations PFAM.

Ainsi, les annotations PFAM ne couvrent qu’environ 30.80% de l’ensemble des protéines étudiées. Cette couverture partielle est une limite centrale de notre évaluation : une absence d’annotation PFAM ne signifie pas une absence de fonction, mais seulement une absence d’information exploitable dans cette source. En conséquence, près de 70% des protéines ne peuvent pas être évaluées directement à partir de PFAM. Cette information est essentielle pour interpréter les métriques biologiques : un cluster peut être structurellement pertinent mais difficile à évaluer si une grande partie de ses protéines n’est pas annotée.

Il est donc nécessaire de distinguer deux situations :

- les paires de protéines évaluable, pour lesquelles les deux protéines possèdent au moins une annotation PFAM ;
- les paires non évaluable, pour lesquelles au moins une protéine ne possède pas d’annotation PFAM.

Nous ne devons pas assimiler une paire non évaluable à une paire incompatible. Une absence d’annotation ne signifie pas nécessairement une absence de similarité fonctionnelle ; elle signifie seulement que l’information disponible ne permet pas de conclure.

Structure multi-label des annotations Une protéine peut posséder une ou plusieurs annotations PFAM. Le problème ne peut donc pas être traité comme une classification monoclasse stricte. Chaque protéine est représentée par un ensemble d’annotations, ce qui correspond à une représentation multi-label.

Le tableau 2 décrit le nombre d’annotations PFAM par protéine annotée.

Statistique	Valeur
Nombre moyen d'annotations PFAM par protéine annotée	1.26
Médiane	1
Premier quartile	1
Troisième quartile	1
Minimum	1
Maximum	35

TABLE 2 – Distribution du nombre d'annotations PFAM par protéine annotée.

La médiane, le premier quartile et le troisième quartile valent tous 1. Cela signifie que la majorité des protéines annotées possèdent une seule annotation PFAM. Cependant, certaines protéines possèdent plusieurs annotations, jusqu'à 35 dans nos données. Il est donc nécessaire de conserver une représentation ensembliste plutôt que de réduire chaque protéine à une unique annotation.

Ce choix est important : si nous ne gardions qu'une seule annotation par protéine, nous perdriions l'information sur les protéines multi-domaines. À l'inverse, utiliser l'ensemble complet des annotations permet de mesurer des similarités partielles entre protéines.

Distribution des annotations PFAM La fréquence des annotations PFAM est très hétérogène. Certaines annotations apparaissent dans un grand nombre de protéines, tandis qu'une grande partie des annotations est rare.

Statistique	Valeur
Nombre d'annotations PFAM distinctes	9 146
Annotation la plus fréquente	PF00069
Proportion couverte par l'annotation la plus fréquente	3.57%
Proportion couverte par les 5 annotations les plus fréquentes	13.16%
Proportion couverte par les 10 annotations les plus fréquentes	17.98%
Proportion couverte par les 20 annotations les plus fréquentes	24.96%
Annotations présentes dans une seule protéine	1 348
Annotations présentes dans moins de 10 protéines	3 638
Annotations présentes dans plus de 1 000 protéines	547
Annotations présentes dans plus de 10 000 protéines	31

TABLE 3 – Distribution des fréquences des annotations PFAM.

L'annotation la plus fréquente, PF00069, correspond au domaine protéine kinase et apparaît dans 83 873 protéines, soit environ 3.57% des protéines annotées PFAM. Elle est donc fréquente, mais elle ne domine pas l'ensemble des données. Les 20 annotations les plus fréquentes couvrent environ 25% des protéines annotées PFAM, ce qui montre l'existence de familles récurrentes. En parallèle, 3 638 annotations apparaissent dans moins de 10 protéines, ce qui met en évidence une longue traîne d'annotations rares.

Cette distribution a deux conséquences pour notre évaluation :

- les annotations très fréquentes peuvent augmenter artificiellement la similarité entre protéines si elles correspondent à des domaines très généraux ;
- les annotations très rares peuvent produire de nombreux scores de similarité nuls, même dans des clusters biologiquement proches mais finement différenciés.

Pour cette raison, les scores basés sur PFAM doivent être interprétés avec prudence : ils donnent une indication utile de l'homogénéité fonctionnelle, mais ne doivent pas être considérés comme une vérité absolue.

Visualisation de la distribution Afin de visualiser cette hétérogénéité, nous utilisons deux représentations complémentaires. La première est un histogramme en échelle logarithmique, qui montre combien d’annotations apparaissent avec une fréquence donnée. La seconde est une courbe de rang, où les annotations sont triées de la plus fréquente à la plus rare.

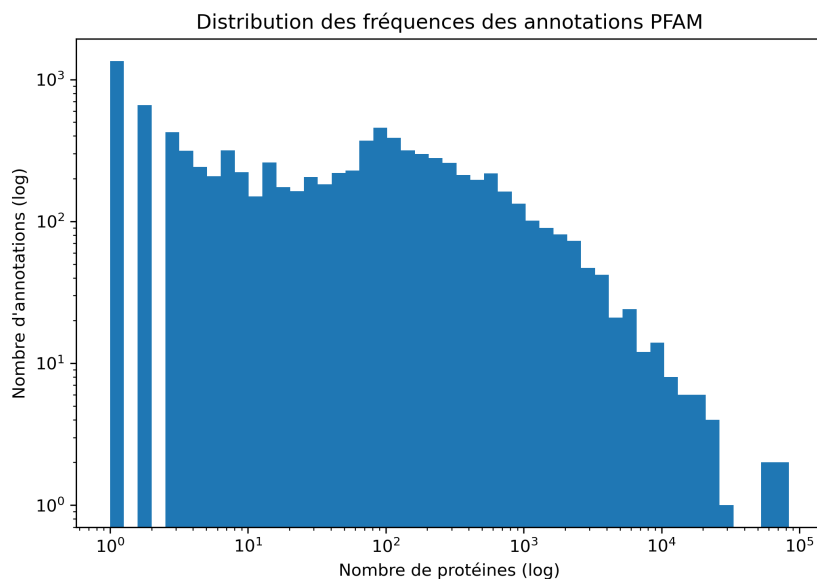


FIGURE 1 – Distribution des fréquences des annotations PFAM en échelle logarithmique.

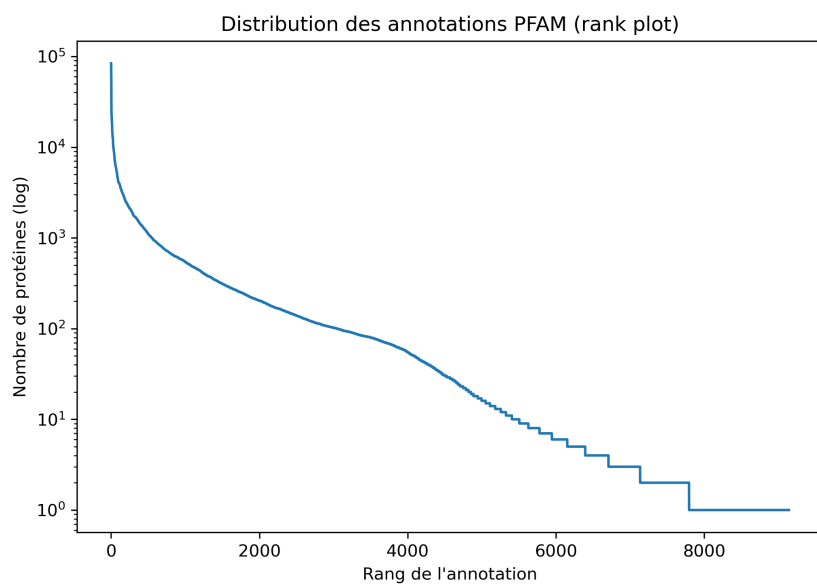


FIGURE 2 – Courbe de rang des annotations PFAM triées par fréquence décroissante.

La figure 1 met en évidence la présence d’un grand nombre d’annotations rares. La figure 2 montre une décroissance rapide des fréquences, caractéristique d’une distribution en longue traîne. Ces deux représentations confirment que l’espace des annotations PFAM est à la fois riche et fortement déséquilibré.

7 Graphes

7.1 Quelques définitions

Soit un graphe $G = (V, E)$ avec :

- V : ensemble de sommets du graphe ;
- E : ensemble d'arêtes du graphe.

Sommet : point du graphe, noté $v \in V$.

Arête : liaison entre deux sommets pour un graphe non-orienté.

Degré : nombre d'arêtes incidentes à un sommet donné.

Chaîne : séquence de sommets et d'arêtes.

Chaîne élémentaire : chaîne qui ne passe pas deux fois par la même arête ni le même sommet.

Graphe connexe : pour tout couple $(u, v) \in V^2$, il existe une chaîne élémentaire entre u et v .

Composantes d'un graphe : si le graphe n'est pas connexe, alors il est divisé en sous-ensembles, donc sous-graphes, qu'on appelle des composantes.

Excentricité ($\epsilon(v)$) : pour un sommet v , c'est la distance maximale entre v et n'importe quel autre sommet du graphe.

$$\epsilon(v) = \max_{u \in V} d(v, u)$$

Rayon ($rad(G)$) : il correspond à l'excentricité minimale parmi tous les sommets du graphe. C'est, en quelque sorte, la distance depuis le sommet le plus « central ».

$$rad(G) = \min_{v \in V} \epsilon(v)$$

Diamètre ($diam(G)$) : c'est la distance maximale entre deux sommets quelconques du graphe (ou l'excentricité maximale).

$$diam(G) = \max_{v \in V} \epsilon(v)$$

Centre : c'est l'ensemble des sommets dont l'excentricité est égale au rayon du graphe.

$$\text{Center}(G) = \{v \in V \mid \epsilon(v) = rad(G)\}$$

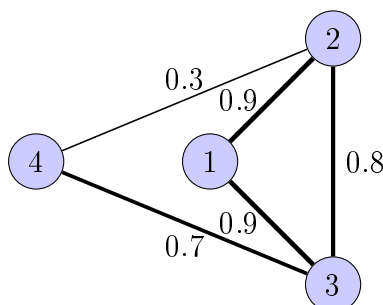
Périphérie : c'est l'ensemble des sommets dont l'excentricité est égale au diamètre du graphe.

$$\text{Periphery}(G) = \{v \in V \mid \epsilon(v) = diam(G)\}$$

Densité (D) : elle mesure le rapport entre le nombre d'arêtes existantes ($|E|$) et le nombre d'arêtes possibles. Pour un graphe non-orienté :

$$D = \frac{2|E|}{|V|(|V| - 1)}$$

7.2 Illustration sur un exemple de graphe non orienté



$G = (V, E)$, graphe non-orienté et pondéré, avec :

- $V = \{1, 2, 3, 4\}$ ($|V| = 4$) ;
- $E = \{\{1, 2\}, \{2, 3\}, \{3, 1\}, \{4, 3\}, \{2, 4\}\}$ ($|E| = 5$) ;

Propriétés du graphe :

(Toujours en prenant *Distance* = 1 - *Similarité*)

- **Connexité** : oui.
- **Distance minimale entre chaque paire de nœuds et excentricité** :

v_1/v_2	1	2	3	4	Excentricité
1	0	0.1	0.1	0.4	0.4
2	0.1	0	0.2	0.5	0.5
3	0.1	0.2	0	0.3	0.3
4	0.4	0.5	0.3	0	0.5

- **Rayon** : 0.3.
- **Diamètre** : 0.5.
- **Centre** : $\{3\}$.
- **Périphérie** : $\{2, 4\}$.
- **Densité** : $D = \frac{2 \times 5}{4 \times 3} = \frac{5}{6}$.

8 Clustering

Dans cette partie, nous verrons les algorithmes de clustering considérés dans notre projet.

8.1 MCL (Markov Clustering Algorithm)

Description : algorithme non supervisé applicable sur un graphe. Le graphe n'a pas besoin d'être connexe et il n'est pas nécessaire de fixer le nombre de clusters avant de lancer l'algorithme [3].

Complexité temporelle : $O(n^3)$ en théorie (pire cas), avec n le nombre de sommets. *La vitesse peut être améliorée par l'élagage, en retirant les arêtes dont le poids est en dessous d'un certain seuil (au lieu d'attendre qu'elles le soient au cours des exécutions de l'algorithme).*

8.1.1 Algorithme

MCL est une simulation de marche aléatoire (random walk) sur le graphe.

Paramètres

- **Le paramètre de puissance e (Expansion)** :
 - Il contrôle l'expansion de la matrice. La matrice sera mise à la puissance e , ce qui simule le fait de faire plusieurs pas sur le graphe donc de changer les probabilités, soit les pas effectués ;
 - Généralement, $e = 2$.
- **Le paramètre d'inflation r** :
 - Après l'expansion, on élève chaque valeur de la matrice à la puissance r puis on normalise. Cela renforce les probabilités fortes et écrase les probabilités faibles. Ainsi, les frontières entre les groupes de protéines deviennent de plus en plus nettes ;

- Généralement, $r \in \{1.1, 5.0\}$. Avec un petit $r \leq 1.5$, on obtient de gros clusters mais peu nombreux. Avec un grand $r \geq 3.0$, on obtient de petits clusters, la granularité du clustering augmente.

Étapes de l'algorithme

1. L'entrée est un graphe non-orienté, avec un paramètre de puissance e , et un paramètre d'inflation r ;
2. Création de la matrice associée au graphe (matrice d'adjacence) ;
3. Normalisation de la matrice ;
4. Calcul de l'expansion de la matrice en appliquant e ;
5. Calcul de l'inflation de la matrice en appliquant r ;
6. Répéter les étapes 5 et 6 jusqu'à convergence, c'est-à-dire que la matrice soit à l'équilibre et que les nombres dans une même colonne aient la même valeur ;
7. Interpréter la matrice résultante pour découvrir les clusters. Regarder chaque ligne de la matrice et noter tous les sommets dont la valeur est différente de 0. Ils seront dans le même cluster.

8.2 Louvain

Communauté : sous-ensemble d'un graphe dans lequel il existe des liens relativement forts, directs, intenses et fréquents. Il y a plus d'arêtes entre les nœuds à l'intérieur de la communauté qu'à l'extérieur.

Modularité : maximise la différence entre le nombre réel d'arêtes dans une communauté et le nombre attendu de ces arêtes (cela étant le nombre d'arêtes si la disposition était faite au hasard). Le nombre attendu d'arêtes peut être exprimé par $\frac{K_c^2}{2m}$, où K_c est la somme des degrés des nœuds de la communauté c et m le nombre total d'arêtes dans le réseau. La modularité est donnée par : $\mathcal{H} = \frac{1}{2m} \sum_c (e_c - \gamma \frac{K_c^2}{2m})$, où $\gamma > 0$ est un paramètre de résolution.

Description : algorithme non supervisé basé sur l'optimisation de la modularité. Il est particulièrement efficace pour les graphes de grande taille.

Bien qu'initialement conçu pour la modularité, l'algorithme de Louvain peut également être utilisé pour optimiser d'autres fonctions de qualité.

Complexité temporelle : $O(n^2)$ en théorie (pire cas), et $O(n \log(n))$ en pratique (sur des données réelles).

8.2.1 Algorithme

Étapes de l'algorithme

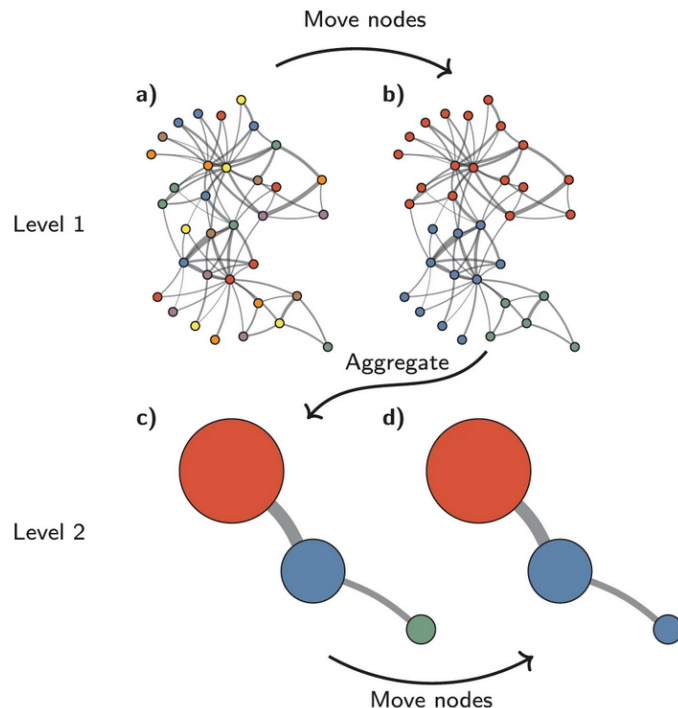


FIGURE 3 – Fonctionnement de l'algorithme Louvain [4]

Cet algorithme part d'une partition initiale où chaque nœud appartient à sa propre communauté.

- (a) Il déplace les nœuds d'une communauté à l'autre pour trouver une nouvelle partition et maximiser le gain de modularité ;
- (b) À partir de cette partition, un réseau agrégé est créé. L'agrégation se fait en construisant un nouveau graphe où les nœuds sont les communautés formées lors de l'optimisation locale. Les arêtes entre ces « super-nœuds » correspondent à la somme des poids des arêtes entre les communautés ;
- (c) L'algorithme déplace alors les nœuds de ce réseau agrégé ;
- (d) Ces étapes d'optimisation locale sont répétées jusqu'à ce qu'il n'y ait plus de gain possible de modularité.

Limites de l'algorithme L'algorithme de Louvain comporte un écueil : **il peut identifier des communautés qui sont intérieurement mal connectées**, un problème qui se produit fréquemment en pratique. En effet, dans l'algorithme de Louvain un nœud peut être déplacé vers une autre communauté alors qu'il servait de pont entre différentes composantes de son ancienne communauté. Le retrait d'un tel nœud de son ancienne communauté déconnecte cette dernière. On pourrait s'attendre à ce que d'autres nœuds de l'ancienne communauté soient alors également déplacés vers d'autres communautés. Cependant, ce n'est pas nécessairement le cas, car ces nœuds peuvent encore être suffisamment fortement connectés à leur communauté, malgré la déconnexion de celle-ci.

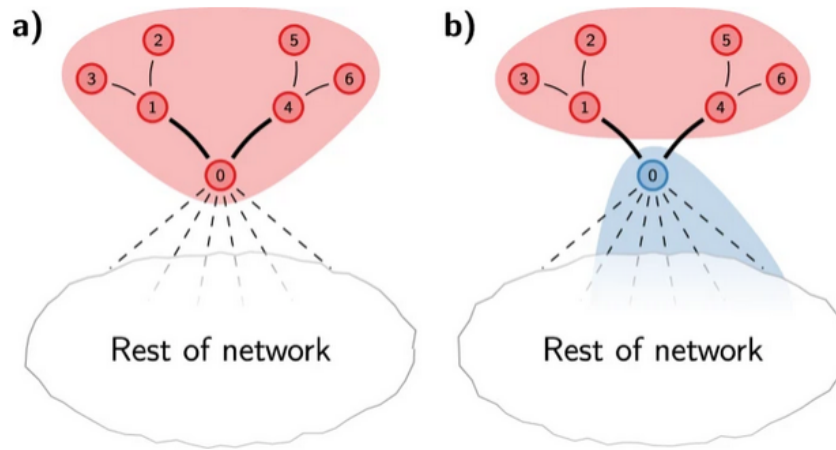


FIGURE 4 – Communautés déconnectées [4]

- (a) Le nœud 0 de la partition rouge est déplacé vers une autre communauté ;
- (b) La communauté rouge devient déconnectée de l'intérieur. Cependant, les nœuds 1 à 6 sont toujours affectés de manière optimale localement et resteront donc dans la communauté rouge.

8.3 Leiden

Description : amélioration directe de Louvain. Elle a été conçue pour corriger le défaut majeur de Louvain : la tendance à générer des communautés mal connectées ou déconnectées à l'intérieur d'elles-mêmes [4]. Ainsi, l'algorithme de Leiden garantit la production de partitions où toutes les communautés sont connectées intérieurement.

Complexité temporelle : $O(n \log(n))$ en théorie, $O(n)$ en pratique.

8.3.1 Algorithme

Leiden ajoute une étape cruciale de « raffinement » pour garantir la cohésion des groupes.

Paramètres

— **Le paramètre de résolution γ :**

- Indique à quel point Leiden doit être « exigeant » sur la densité interne des groupes. Il stipule de l'importance accordée aux connexions internes par rapport à ce que l'on attendrait d'un graphe aléatoire ;
- Si γ est faible, l'algorithme acceptera des clusters plus vastes et moins denses ;
- Si γ est élevé, l'algorithme cherchera des zones de densité très forte, il sera plus stricte. Les clusters seront plus petits et très soudés, la granularité sera forte.

Étapes de l'algorithme

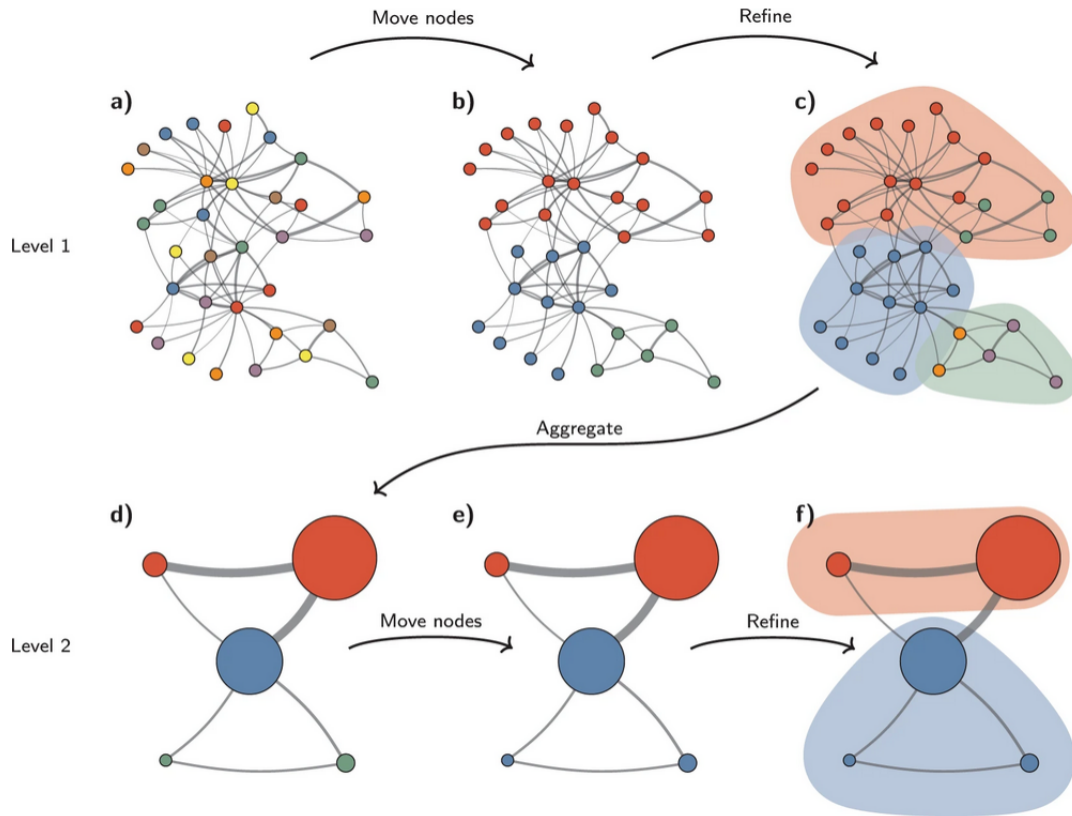


FIGURE 5 – Fonctionnement de l'algorithme Leiden [4]

L'algorithme de Leiden part d'une partition unique.

- (a) Il déplace les nœuds individuellement d'une communauté à une autre pour trouver une partition ;
- (b) La partition est raffinée. Le raffinement est la nouveauté de Leiden par rapport à Louvain. Au lieu d'agréger directement, l'algorithme examine les communautés formées et les subdivise si elles ne sont pas assez denses ou connectées. Cela garantit que les communautés finales soient structurellement robustes ;
- (c) Le réseau est agrégé ;
- (d) Est créé à partir de cette partition affinée, la partition initiale servant à établir la partition du réseau agrégé ;
- (e) L'algorithme déplace ensuite les nœuds individuellement dans le réseau agrégé ;
- (f) Dans ce cas, l'affinage ne modifie pas la partition.

Ces étapes sont répétées jusqu'à ce qu'aucune amélioration supplémentaire ne soit possible.

8.4 Comparaison structurelle des partitions

Afin de comparer les partitions produites à partir de nos différents algorithmes de clustering, nous pouvons regarder les résultats de leurs métriques d'évaluation, mais également comparer le contenu des clusters.

Pour cela, il existe plusieurs méthodes.

8.4.1 L'Index de Rand (RI)

Cette mesure permet de comparer deux partitions π_1 et π_2 dans l'ensemble E .

Principe : regarde toutes les paires de sommets dans π_1 , puis regarde lesquelles sont aussi dans π_2 .

Algorithme

L'indice de Rand (RI) est défini par :

$$RI(\pi_1, \pi_2) = \frac{a + d}{a + b + c + d}$$

C'est-à-dire la proportion des paires d'éléments qui sont conjointement groupées ou conjointement séparées.

Avec :

- a : le nombre de paires présentes dans π_1 et π_2 ;
- b : le nombre de paires présentes dans π_1 mais pas dans π_2 ;
- c : le nombre de paires présentes dans π_2 mais pas dans π_1 ;
- d : le nombre de paires de E absentes de π_1 et π_2 .

8.4.2 Indice de Rand Ajusté (ARI)

$$ARI = \frac{(RI - \text{Expecté_}RI)}{(\max(RI) - \text{Expected_}RI)}$$

- 1.0 : les clusters sont identiques ;
- 0.0 : la labellisation a été faite aléatoirement indépendamment du nombre de clusters ;
- -0.5 : les clusters sont totalement différents.

9 Métriques d'évaluation de clustering

En général, les métriques d'évaluation sont calculées avec les **distances entre les points** (un point = un sommet de notre graphe, donc une protéine). Cependant, ici nous n'avons pas de distance. La seule mesure qui nous permet de comparer nos sommets entre eux est le **poids de l'arête qui les relie**.

Nous pouvons donc **remplacer la notion de distance par** : $d = 1 - w_{ij}$, avec w_{ij} le poids entre les sommets i et j , normalisé entre 0 et 1.

Ainsi, lorsque d augmente, w_{ij} diminue, donc la similarité entre les deux protéines diminue.

Principales caractéristiques attendues du clustering

- **Homogénéité d'un cluster** : un cluster est homogène si ses points sont proches les uns des autres, c'est-à-dire s'ils présentent des caractéristiques ou des attributs similaires.
- **Séparabilité entre les clusters de la partition** : les clusters d'une partition doivent être distants les uns des autres, donc avoir le moins possible de caractéristiques en commun.

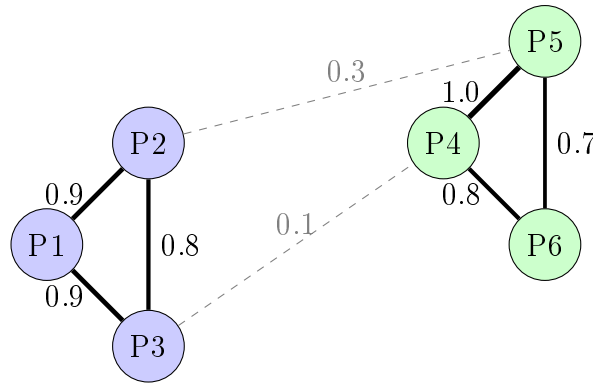
Que faire quand les scores obtenus à partir des métriques d'évaluation sont trop faibles ?

Si les scores sont trop faibles, il faudra reconsidérer la méthode employée et essayer d'optimiser les paramètres du modèle, voire changer de modèle.

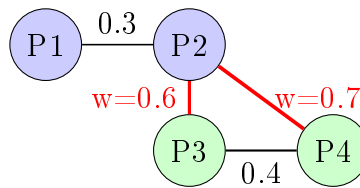
Néanmoins, si seule une poignée de clusters a des scores internes faibles, alors il faudra les recalculer. Dans ce cas, on peut passer au modèle un nouveau graphe composé uniquement de ces clusters trop faibles. Ou bien, nous pourrions traiter chaque cluster un par un en le redivisant en sous-clusters.

9.1 Exemples de partitions

9.1.1 Partition idéale - forte homogénéité et séparabilité



9.1.2 Mauvaise partition - hétérogénéité et faible séparabilité



9.2 Mesures d'homogénéité sur une partition P

9.2.1 Co-distance de P

À minimiser. Ensemble : $[0, +\infty[$.

$$codist(P) = \sum_{k=1}^K \sum_{\{x, x'\} \subseteq C_k} d(x, x')$$

Avec C_k le k ème cluster de la partition P .

Somme les distances des arêtes intra-cluster pour chaque cluster de la partition.

Valeur :

- **Faible** : dans un même cluster, les points sont très proches entre eux donc partagent de nombreuses caractéristiques communes.
- **Élevée** : les clusters de la partition sont étalés, certains de leurs points sont reliés par des arêtes de poids faible.

9.3 Mesures de séparabilité des clusters de la partition

9.3.1 Coupe

À minimiser. Ensemble : $[0, +\infty[$.

Somme des poids des arêtes reliant un cluster C au reste de la partition (on veut couper le moins possible d'arêtes fortes).

$$Cut(C_1, C_2) = \sum_{i \in C_1, j \in C_2} w_{ij}$$

Avec C_1 et C_2 , deux clusters, et w le poids.

Mesure le coût de la séparation des clusters.

Valeur :

- **Faible** : on coupe peu de « poids », donc les clusters sont bien séparés.
- **Élevée** : on coupe beaucoup de poids forts, les clusters sont mal séparés.

9.3.2 Coupe Normalisée

À minimiser. Ensemble : $[0, 2]$.

Divise la valeur de la coupe par le volume total du cluster (somme de tous les poids de ses arêtes).

Elle permet de ne pas favoriser les petits clusters.

$$NCut(C_1, C_2) = \frac{Cut(C_1, C_2)}{Vol(C_1)} + \frac{Cut(C_1, C_2)}{Vol(C_2)}$$

Avec C_1 et C_2 , deux clusters, et w le poids.

$$Vol(C) = \sum_{i,j \in C} w_{ij}$$

Valeur :

- **Faible** : on coupe peu de « poids », donc les clusters sont bien séparés.
- **Élevée** : on coupe beaucoup de poids forts, les clusters sont mal séparés.

9.4 Mesures basées sur les annotations

Mesure de similarité entre deux protéines Pour comparer deux protéines annotées, nous utilisons l'indice de Jaccard entre leurs ensembles d'annotations PFAM. Pour deux protéines i et j , on note A_i et A_j leurs ensembles d'annotations PFAM. La similarité de Jaccard est définie par :

$$J(A_i, A_j) = \frac{|A_i \cap A_j|}{|A_i \cup A_j|}.$$

Ce score vaut :

- 1 si les deux protéines possèdent exactement les mêmes annotations PFAM ;
- 0 si elles ne partagent aucune annotation PFAM ;
- une valeur intermédiaire si elles partagent seulement une partie de leurs annotations.

Le schéma 6 illustre ce calcul sur un exemple simple.

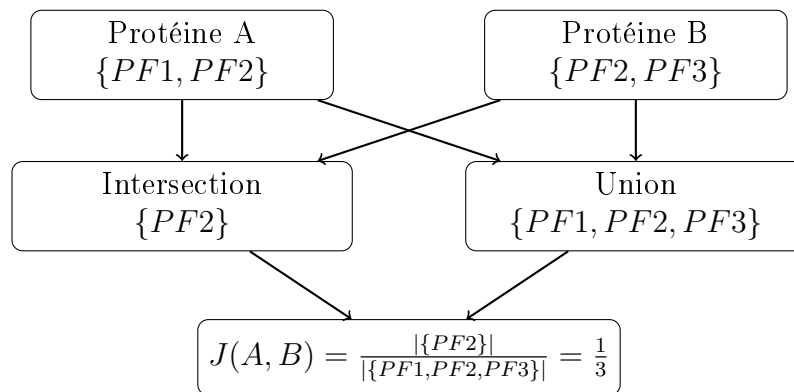


FIGURE 6 – Exemple de calcul du score de Jaccard entre deux protéines représentées par leurs ensembles d'annotations PFAM.

L'utilisation du Jaccard est adaptée à notre problème car elle prend en compte le caractère multi-label des annotations. Une simple égalité d'étiquettes ne serait pas suffisante, puisqu'une protéine peut partager une partie de ses domaines avec une autre sans avoir exactement le même ensemble d'annotations.

Définition des paires compatibles, incompatibles et non évaluables Pour l'évaluation des clusters, nous considérons les paires de protéines à l'intérieur d'un même cluster. Trois cas sont distingués :

- **Paire compatible** : les deux protéines possèdent au moins une annotation PFAM commune. Le score de Jaccard est strictement positif.
- **Paire incompatible** : les deux protéines sont annotées, mais ne partagent aucune annotation PFAM. Le score de Jaccard vaut 0.
- **Paire non évaluable** : au moins une des deux protéines ne possède pas d'annotation PFAM. Le score de Jaccard n'est pas calculé.

Cette distinction est essentielle. Une paire non évaluable ne doit pas être considérée comme incompatible, car l'absence d'annotation ne permet pas de conclure biologiquement. Nous comptabilisons donc séparément les paires vides afin de mesurer le degré d'information disponible dans chaque cluster.

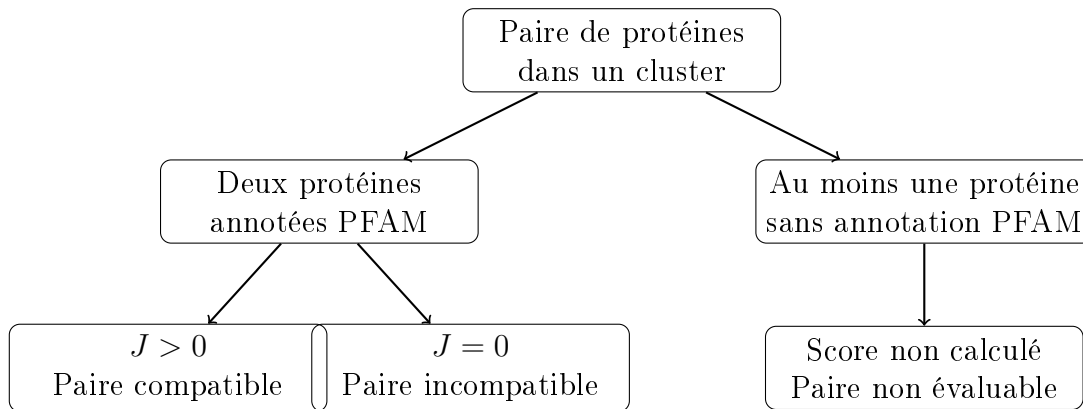


FIGURE 7 – Classification des paires de protéines utilisées pour l'évaluation des clusters.

Stockage des compatibilités entre annotations Afin de faciliter le calcul des incompatibilités entre protéines, nous avons d'abord étudié les cooccurrences entre annotations PFAM. Deux annotations sont considérées comme compatibles si elles apparaissent ensemble dans au moins une même protéine. À l'inverse, deux annotations sont considérées comme incompatibles si elles n'ont jamais été observées ensemble dans une même protéine. Cette structure est utilisée notamment pour vérifier rapidement si deux ensembles d'annotations peuvent être fusionnés sans introduire d'association jamais observée dans les données.

Sur les 9 146 annotations PFAM distinctes, il existe :

$$\frac{9146 \times 9145}{2} = 41\,820\,085$$

paires possibles d'annotations. Parmi celles-ci, seules 62 294 paires sont effectivement observées ensemble dans au moins une protéine, soit environ 0.15% des paires possibles. Les paires non observées représentent donc 99.85% de l'espace des paires.

Type de paire d'annotations	Nombre
Paires possibles	41 820 085
Paires compatibles observées	62 294
Paires non observées	41 757 791
Proportion de paires compatibles	0.15%
Proportion de paires non observées	99.85%

TABLE 4 – Cooccurrences observées entre annotations PFAM.

Cette forte sparsité justifie de stocker les paires compatibles plutôt que les paires incompatibles. En effet, stocker explicitement les incompatibilités reviendrait à conserver presque toutes les paires possibles, ce qui serait inutilement coûteux. En pratique, nous construisons donc un dictionnaire des compatibilités observées :

$$\text{compatible}[\text{label}] = \{\text{annotations observées avec } \text{label}\}.$$

Deux annotations sont ensuite considérées comme incompatibles si elles sont différentes et si elles ne sont pas présentes dans ce dictionnaire de compatibilité.

Cette définition doit cependant être interprétée avec prudence : une absence de cooccurrence dans les données ne signifie pas nécessairement une incompatibilité biologique absolue. Elle indique seulement que cette association n'a pas été observée dans notre jeu de données. Nous utilisons donc cette notion comme un critère opérationnel pour repérer des clusters potentiellement hétérogènes, et non comme une vérité biologique définitive.

Évaluation d'un cluster Pour chaque cluster C , nous calculons plusieurs statistiques complémentaires :

- le nombre total de protéines du cluster ;
- le nombre et la proportion de protéines possédant au moins une annotation PFAM ;
- le nombre total de paires possibles dans le cluster ;
- le nombre de paires évaluables ;
- la proportion de paires non évaluables ;
- la moyenne et la médiane des scores de Jaccard sur les paires évaluables ;
- la proportion de paires incompatibles parmi les paires évaluables.

Ces métriques permettent de distinguer plusieurs types de clusters :

- des clusters bien annotés et homogènes, contenant beaucoup de protéines annotées et peu de paires incompatibles ;
- des clusters hétérogènes, contenant des protéines annotées mais avec de nombreux scores de Jaccard nuls ;
- des clusters peu informatifs, contenant trop peu de protéines annotées pour être évalués de manière fiable.

L'objectif n'est donc pas seulement d'obtenir un score moyen global, mais d'identifier précisément les clusters problématiques. Cela répond à une limite importante des métriques globales : une moyenne élevée peut masquer l'existence de certains clusters très incohérents, tandis qu'une moyenne faible peut être due à un grand nombre de clusters non évaluables.

Utilisation pour comparer les partitions Les métriques basées sur PFAM sont calculées pour les partitions obtenues avec Leiden et MCL, ainsi que pour différentes valeurs du paramètre α dans la combinaison :

$$S_{\text{final}} = \alpha S_{\text{BLAST}} + (1 - \alpha) S_{\text{embed}}$$

Le but est de déterminer si l'ajout des embeddings améliore l'homogénéité fonctionnelle des clusters par rapport au graphe BLAST seul. Plus précisément, nous cherchons à observer si certaines valeurs de α permettent :

- d'augmenter la similarité moyenne de Jaccard ;
- de diminuer la proportion de paires incompatibles ;
- de réduire le nombre de clusters non évaluables ;
- ou de mieux structurer les protéines annotées et non annotées.

Cette analyse est complémentaire des métriques classiques de clustering. En effet, deux partitions peuvent avoir des scores structurels proches tout en différant dans leur cohérence fonctionnelle. Les annotations PFAM permettent donc d'évaluer si les modifications apportées aux poids du graphe par les embeddings ont un effet biologique observable.

Limites de l'approche Cette méthode présente néanmoins plusieurs limites. Premièrement, toutes les protéines ne possèdent pas d'annotation PFAM, ce qui réduit la couverture de l'évaluation. Deuxièmement, la distribution très déséquilibrée des annotations peut influencer les scores : les annotations fréquentes peuvent créer des similarités peu spécifiques, tandis que les annotations rares peuvent conduire à des scores nuls même entre protéines proches. Enfin, PFAM ne capture pas toute la fonction biologique d'une protéine ; il s'agit d'une approximation basée sur les domaines détectés.

Malgré ces limites, cette approche fournit une évaluation plus interprétable que les métriques purement structurelles, car elle relie directement la composition des clusters à des informations biologiques indépendantes du graphe utilisé pour le clustering.

10 Mesures de la qualité de la prédiction d'annotation

10.1 Précision

La précision (valeur prédictive positive) est la proportion des items pertinents parmi l'ensemble des items proposés.

$$P = \frac{TP}{TP + FP}$$

avec :

- TP : vrais positifs
- FP : faux positifs
- TN : faux négatifs
- FN : faux négatifs

Ce qui peut également s'écrire :

$$\text{précision}_i = \frac{\text{nb de protéines correctement attribuées à l'annotation a}}{\text{nb de protéines totales attribuées à l'annotation a}}$$

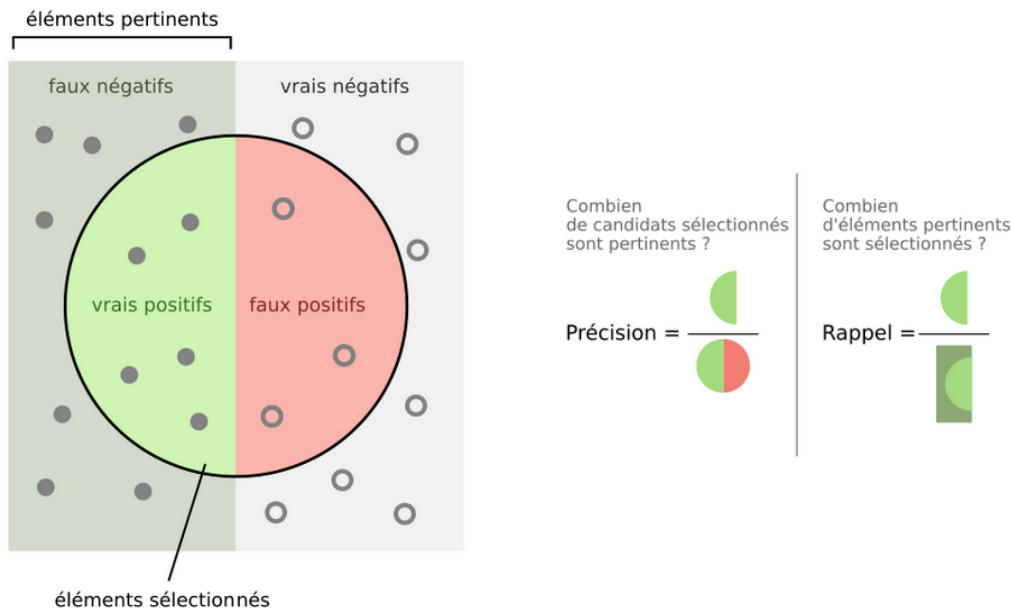
10.2 Rappel

Le Rappel (sensibilité) est la proportion des items pertinents proposés parmi l'ensemble des items pertinents.

$$R = \frac{TP}{TP + FN}$$

Ce qui peut également s'écrire :

$$\text{rappel}_i = \frac{\text{nb de protéines correctement attribuées à l'annotation } a}{\text{nb de protéines possédant vraiment l'annotation } a}$$



10.3 F1-score

Une mesure qui combine la précision et le rappel est leur moyenne harmonique, nommée F1-mesure ou F1-score. C'est un F-score à l'équilibre, c'est-à-dire avec $\beta = 1$, qui donne la même importance au rappel et à la précision.

$$F1 = 2 \cdot \frac{(\text{précision} \cdot \text{rappel})}{(\text{précision} + \text{rappel})}$$

Nous calculons également la **couverture de prédiction**, définie comme la proportion de protéines du jeu de test recevant au moins une annotation prédite. Cette mesure est essentielle dans notre cas : une méthode peut obtenir une précision élevée simplement parce qu'elle prédit très peu d'annotations. La couverture permet donc de distinguer une méthode fiable mais trop conservatrice d'une méthode réellement exploitable à grande échelle.

Enfin, nous distinguons deux niveaux de scores :

- les scores **micro**, calculés globalement sur l'ensemble des annotations prédites et réelles ;
- les scores **macro**, calculés en moyennant les scores par protéine ou par label, afin d'évaluer la stabilité des performances selon les protéines ou les annotations.

Dans la suite, nous privilégierons le F1-score micro pour comparer les méthodes, car il mesure le compromis global entre annotations correctes retrouvées et annotations incorrectement ajoutées.

11 Propagation des annotations

Annotations à propager au sein d'un même cluster

1. **Lister les annotations majoritaires** : celles dont la proportion dépasse 30% des annotations du cluster
2. **Repérage des paires incompatibles**
3. **Si les annotations A et B sont incompatibles** :
 - (a) $\%(A) > \%(B)$: garder A, supprimer B
 - (b) $\%(B) > \%(A)$: garder B, supprimer A
 - (c) $\%(A) = \%(B)$: par sécurité, on supprime les deux

Seuil de propagation choisi : 0.7

Différents cas de la propagation d'annotations majoritaires

Prenons une protéine P1 à propager à P2.

- **Annotations de P1** : A, B, C.
- **Annotations de P2** : D, E.

A et B sont des annotations majoritaires, ce sont donc elles que nous allons propager.

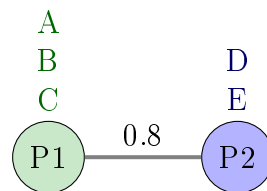


FIGURE 8 – État de départ

Cas 1 : P1 et P2 ont des annotations 100% compatibles

On procède donc à l'union des annotations de P1 et P2.

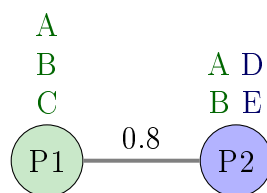


FIGURE 9 – Union des annotations sur P2

Cas 2 : incompatibilité entre deux annotations de P1 et P2

A et D sont incompatibles.

On n'effectue pas de propagation.

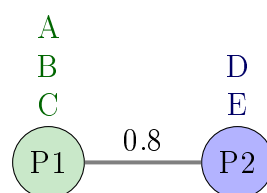
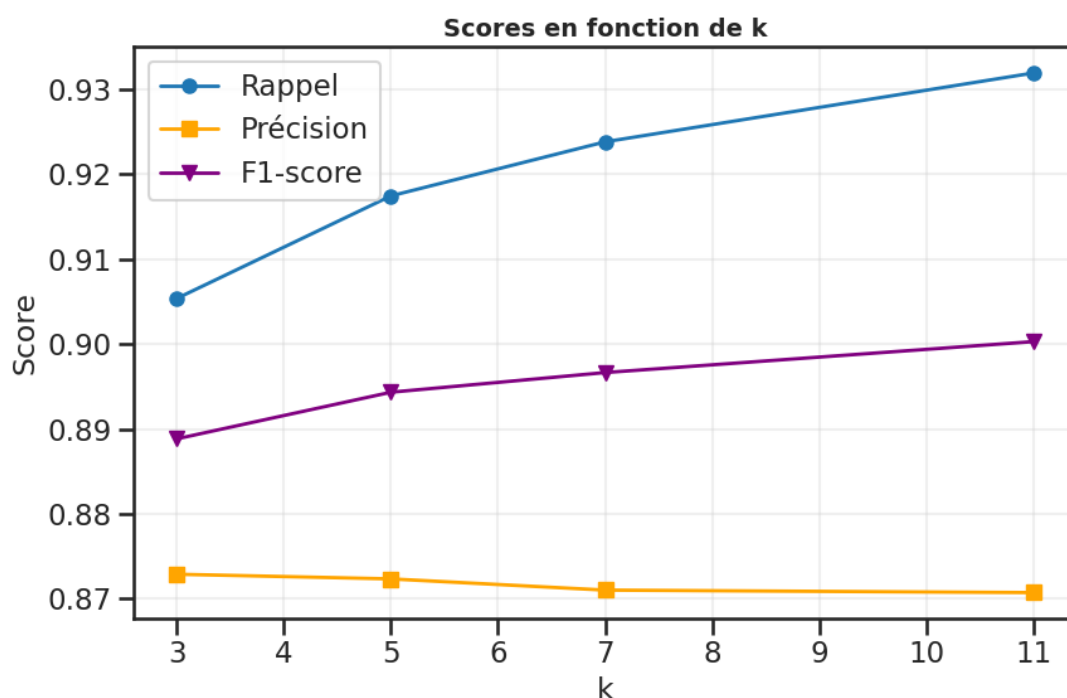


FIGURE 10 – Pas de propagation

12 L'algorithme des k plus proches voisins (kNN) comme standard d'annotation fonctionnelle

Dans le domaine de la bio-informatique, la prédiction de fonctions de protéines repose historiquement sur le principe du transfert d'annotation par homologie. Deux protéines partageant une forte similarité de séquence ou de structure sont issues d'un ancêtre commun et ont, par conséquent, des fonctions biologiques similaires. L'algorithme des k plus proches voisins s'est naturellement imposé comme l'un des standards de la littérature.

Dans le cadre de ce projet, le kNN classique sert de point de référence à nos approches utilisant des algorithmes de clustering. L'algorithme a été exécuté sur le graphe BLAST de nos données en masquant temporairement les annotations d'un sous-ensemble de test qui représente 20% des données, et a donc été entraîné sur les 80% restants. De plus, nous avons testé différentes valeurs de k pour voir leur impact sur les résultats : $k \in \{3, 5, 7, 11\}$



Les résultats sont très bons. Ils nous permettent de valider topologiquement le graphe BLAST de nos données. La précision varie de 87,3% à 87,1%, le rappel de 90,5% à 93,2%, le f1-score est à l'équilibre entre les deux. Le meilleur k est $k = 11$, avec un F1-score de 90,0%.

Si le kNN a de si bons résultats, pourquoi ne pas le garder au lieu d'explorer d'autres approches ? Le kNN a de très bonnes performances, comme nous pouvons le constater avec ces métriques. Néanmoins, il a de trop grands écueils qui ne le rendent pas viable pour un problème à grande échelle.

Inconvénients du kNN

- **Complexité temporelle** : de l'ordre de $O(M \times V \times \log(V))$ pour notre problème, avec :
 - M : le nombre de protéines à prédire (taille de l'ensemble de test) ;
 - V : le degré de la protéine à prédire, soit son nombre de voisins ;
 - $\log(V)$: coût du tri des voisins de la protéine à prédire (effectué pour extraire les k premières).

- **Trop coûteux à grande échelle** : la complexité temporelle du kNN montre que pour un problème à grande échelle, l'algorithme sera bien trop gourmand niveau temps, sans parler de son poids en RAM ;
- **Sensible aux valeurs aberrantes** : s'il y a une protéine à valeur aberrante dans le voisinage (notation aberrante, ici), l'arête qui la relie à la protéine à prédire aura beau avoir un poids faible, s'il n'y a pas d'autre voisin, alors le kNN la sélectionnera pour prédire l'annotation. Nous évitons ce problème avec notre clustering puisque nous ne propageons que les annotations majoritaires dont la proportion est supérieur à 30% des annotations dans le cluster ;
- **Ne peut voir que ce qui est connu** : le kNN est un algorithme simple qui ne s'arrête qu'aux k plus proches voisins. Il ne pourra pas explorer plus loin que cela, donc si une protéine de test n'a pas de relation avec une autre protéine annotée d'entraînement, elle n'aura jamais d'annotation. Contrairement à nos algorithmes de clustering qui font de la propagation sur plusieurs itérations, évitant ce genre de problèmes et propageant les annotations plus loin.

13 Première approche

Création d'un nouveau graphe en effectuant une combinaison linéaire pondérée entre le graphe de similarité BLAST et le graphe de la similarité cosinus des embeddings.

Combinaison linéaire pondérée : $Score_{final} = \alpha \times S_{blast} + (1 - \alpha) \times S_{cos}$ avec $\alpha \in [0, 1]$.

13.1 Étapes

1. Combinaison linéaire pondérée entre le score BLAST et la similarité cosinus des embeddings pour recalculer le poids des arêtes ;
2. Clustering avec MCL et Leiden. Cela permet de tester plusieurs algorithmes de clustering puis de comparer leurs résultats ;
3. Calcul et analyse des métriques de clustering pour juger de la qualité de la partition obtenue (la co-distance pour l'homogénéité et la coupe normalisée pour la séparabilité) ;
4. Propagation des annotations au sein de la partition ;
5. Calcul et analyse des mesures de la qualité de la prédiction des annotations (f1-score, rappel, précision).

14 Deuxième approche

Aligner les clusters obtenus à partir du réseau de similarité et ceux obtenus à partir des embeddings.

14.1 Étapes générales

1. Obtention de la partition obtenue avec le graphe de similarité BLAST (calculée dans la première approche) ;
2. Obtention de la partition obtenue avec le graphe de similarité cosinus des embeddings (calculée dans la première approche) ;
3. Combinaison des deux partitions.

14.2 Combinaison des deux partitions

Boucle principale

1. Propagation des annotations dans la partition de BLAST ;
2. Pour toute protéine, reporter son annotation dans BLAST aux embeddings ;
3. Propagation des annotations dans la partition des embeddings ;
4. Pour toute protéine, reporter son annotation dans les embeddings à BLAST.

Condition d'arrêt

La boucle principale s'arrêtera lorsque la convergence sera atteinte, c'est-à-dire lorsque ni BLAST ni les embeddings n'auront de nouvelle protéine annotée d'une itération à l'autre.

Pour reporter les annotations d'un cluster à l'autre, on utilise la même logique que dans la propagation d'annotations.

14.2.1 Fusion finale des partitions après la convergence

Une fois la condition d'arrêt atteinte, l'algorithme a convergé vers un état stable. Chaque protéine possède un ensemble d'annotations dans la partition BLAST ($P1_B$) et un ensemble dans la partition des Embeddings ($P1_E$). Le système de filtrage strict appliqué durant les itérations garantit qu'aucun conflit majeur n'a été propagé activement.

Cependant, au moment de livrer la prédiction finale, deux scénarios se présentent :

Cas 1 : compatibilité

Si les annotations de $P1_B$ et $P1_E$ sont compatibles (ou identiques), leur union donne l'annotation prédite finale par l'algorithme pour $P1$. Les deux approches se sont enrichies mutuellement.

Cas 2 : incompatibilité

Dans de rares cas, si $P1$ possédait déjà des annotations initiales contradictoires dans l'ensemble d'entraînement ou si elle est restée isolée à la frontière de deux clusters, $P1_B$ et $P1_E$ peuvent présenter une incompatibilité qui n'a pas pu être résolue par la propagation.

Pour résoudre ce problème, nous évaluons la force de l'ancrage de la protéine dans ses communautés respectives en calculant la somme des poids de ses connexions directes intra-cluster pour les protéines qui partagent le même sous-groupe fonctionnel.

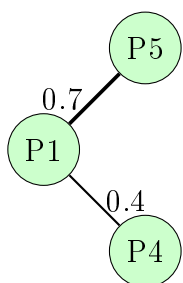


FIGURE 11 – Connexions de P1 dans son cluster BLAST

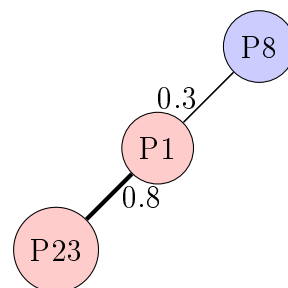


FIGURE 12 – Connexions de P1 dans son cluster Embeddings

Pour trancher, on compare l'adhérence locale de la protéine dans chacun de ses clusters :

$$S_{BLAST} = 0.7 + 0.4 = 1.1$$

$S_{Embeddings} = 0.8$ (l'arête vers le nœud $P8$ n'est pas comptée car le nœud n'appartient pas au même sous-groupe fonctionnel)

$$S_{BLAST} > S_{Embeddings}$$

BLAST a gagné, c'est donc son ensemble d'annotations que l'on va sélectionner dans le résultat final.

En cas d'égalité parfaite des scores ($S_{Embeddings} = S_{BLAST}$), la protéine est déclarée indéterminée pour éviter de propager la moindre incompatibilité.

15 Protocole expérimental

15.1 Création des Embeddings

15.1.1 Choix du modèle

Afin de créer les vecteurs Embeddings pour nos protéines, nous avons voulu utiliser le modèle **ProteinBERT** [1].

Cependant, nous n'avons tout d'abord pas pu réaliser la création des Embeddings en raison du manque d'un fichier essentiel du modèle, qui était absent sur le GitHub et devait être téléchargé directement depuis les serveurs de l'Université Hébraïque de Jérusalem. Ces serveurs se trouvent être hors service (TimeoutError).

Nous avons donc envoyé un mail à Nadav Brandes, l'un des auteurs du modèle et voici sa réponse : "For technical questions/requests you should pose an Issue on our GitHub page (I'd also recommend searching through existing Issues, including closed ones)". Cependant, nous avons déjà cherché et trouvé d'autres personnes ayant eu des problèmes similaires. Par exemple : "[I have an issue to download the pkl \[...\] No such directory 'users/nadavb/protein_bert'](#)" en réponse, Nadav Brades confirme que certains serveurs bloquent l'accès. Néanmoins, une solution a finalement été trouvée : le fichier manquant a été déposé sur un miroir du nom de Zenodo par des utilisateurs de ProteinBERT.

Entre-temps, afin de ne pas bloquer le projet, nous avons opté pour **ESM-2** (Evolutionary Scale modeling) [2], une architecture BERT similaire créée par Meta. Elle utilise un transformeur (architecture de type BERT) pré-entraîné. Elle prend en entrée des séquences FASTA, comme **ProteinBERT** et renvoie des embeddings de dimension 320 (choix de la version 8M). Nous avons donc comparé les deux modèles ci-dessous.

15.1.2 Descriptions de ProteinBERT et de ESM-2

ProteinBERT

- **Type d'architecture** : Transformer/BERT.
- **Utilisation** : prédiction de **fonctions protéiques**. Néanmoins, dans ce projet nous n'utilisons pas directement la prédiction de fonctions mais seulement les embeddings qu'il crée afin de les associer à notre graphe de similarité BLAST. Deux protéines qui ont la même fonction auront des embeddings similaires.
- **Entraînement** : entraîné sur 106 millions de protéines (représentant l'ensemble des protéines connues) sur deux tâches simultanées. La première est la **modélisation bidirectionnelle du langage naturel des séquences protéiques** (une approche bidirectionnelle signifie que le modèle apprend à prédire un acide aminé en regardant simultanément ce qui se trouve à sa gauche et à sa droite dans la séquence). La seconde est la **prédiction des annotations Gene Ontology (GO)**, qui capture diverses fonctions protéiques. Les annotations GO constituent un ensemble d'environ 45 000 termes, validés manuellement et définis au niveau de la protéine entière, couvrant l'ensemble de l'espace protéique chez tous les organismes. Elles couvrent les fonctions moléculaires, les processus biologiques et les localisations subcellulaires.
- **Taille des vecteurs produits** : 15 530 dimensions. L'embedding de ProteinBERT est délibérément sur-dimensionné pour inclure un large spectre de prédictions fonctionnelles, de nombreuses n'étant utilisées que dans des cas spécifiques.
- **Algorithme** : contrairement aux Transformers classiques, ProteinBERT sépare les représentations locales (au niveau des caractères) et globales (au niveau de la séquence entière), ainsi que les entrées et les sorties, permettant ainsi un traitement multitâche rigoureux des tâches locales et globales. Il va donc réaliser une moyenne

interne : il regarde chaque acide aminé de la séquence et se base sur leur position pour générer une représentation, puis il fait une moyenne pondérée de ces 1024 positions pour former un vecteur de 15 530 dimensions.

- **Prise en compte des mutations des protéines** : deux protéines qui étaient identiques au départ et ont subi des mutations auront des embeddings similaires. En effet, ProteinBERT a été entraîné à prédire des acides aminés manquants d'une protéine, et pour cela a dû apprendre quelles mutations sont acceptables et impossibles. Ainsi, il comprendra que deux protéines mutées sont en réalité très proches et leur donnera des embeddings similaires.

ESM-2

- **Type d'architecture** : Transformer/BERT.
- **Utilisation** : prédiction de la **structure 3D** des protéines. Il capture des caractéristiques essentielles pour des tâches telles que la classification ou la prédiction d'interactions, en effet la fonction et les interactions des protéines dépendent souvent de régions locales spécifiques. Il peut également créer des embeddings qui capturent la "grammaire" biologique.
- **Entraînement** : entraîné sur des millions de protéines, sa tâche principale a été de deviner une partie d'une protéine. Par exemple, on lui donne une protéine dont 15% est caché, et il doit prédire ce qu'il manque. Cela lui permet d'apprendre des représentations de séquences pertinentes.
- **Taille des vecteurs produits** : 320 dimensions avec la version 8M.
- **Algorithme** : traite les séquences d'acides aminés comme du texte, en codant chaque acide aminé par son type et sa position. Il supporte des séquences de longueur maximale de 2048 mais consomme beaucoup de mémoire à ce stade donc en général, on utilise des séquences de 1024 acides aminés.
- **Prise en compte des mutations des protéines** : comme pour ProteinBERT, deux protéines qui étaient identiques au départ et ont subi des mutations auront des embeddings similaires. En effet, ESM-2 a été entraîné à prédire des acides aminés manquants d'une protéine, et pour cela a dû apprendre quelles mutations sont acceptables et impossibles. Ainsi, il comprendra que deux protéines mutées sont en réalité très proches et leur donnera des embeddings similaires. Néanmoins, il restera moins efficace que ProteinBERT étant donné qu'il est plus sensible à la structure locale des protéines.

15.1.3 Méthode de création des embeddings

Nous avons les données suivantes :

- **Nombre de protéines étudiées** : 7 633 132 protéines
- Nombre de protéines dont la longueur est **supérieure à 512** : 1 128 276 protéines
- Nombre de protéines dont la longueur est **supérieure à 1024** : 182 936 protéines

Afin de créer les embeddings avec **ESM-2** et **ProteinBERT**, nous avons effectué les calculs sur les GPU de la fac. Étant donné que nous sommes limitées en mémoire, le nombre maximal d'acides aminés pouvant être traités à la fois est de 1024.

Au départ, nous pensions imposer une limite de 512 aux protéines afin que les calculs ne prennent pas trop de temps, mais cela limitait beaucoup nos résultats. En effet, 14,78% de nos données ont une longueur supérieure à 512. Nous avons donc décidé d'**augmenter la limite à 1024**, mais il restait encore 182 936 protéines qui ne seraient pas totalement prises en compte, ce qui ne fait que 0,02%.

Pour **ESM-2**, nous avons adopté la **stratégie de la fenêtre glissante**. Pour une même protéine, cela permet de générer un embedding pour chaque tranche de 1024 puis

de fusionner les résultats en prenant la moyenne arithmétique afin d'obtenir l'embedding de la protéine.

Pour **ProteinBERT**, nous avons tout simplement coupé la protéine après 1024 acides aminés, étant donné que la perte serait très légère car peu de protéines dépassent ce nombre. Le modèle génère donc un seul embedding pour ce bloc de 1024. De plus, nous avons dû ajouter une étape supplémentaire à notre création d'embeddings. En effet, comme mentionné plus haut, ProteinBERT crée des vecteurs de 15 530 dimensions. Un nombre aussi conséquent de dimensions résulte en des fichiers très volumineux et des calculs longs, pour un apport faible. En effet, comme mentionné plus haut, les embeddings générés par ProteinBERT sont délibérément sur-dimensionnés pour inclure un large spectre de prédictions fonctionnelles. Nos protéines provenant de différents génomes de la même espèce, leurs embeddings auront de nombreuses dimensions en commun. Ainsi, en réduisant la dimension à l'aide d'algorithmes dédiés comme le PCA (Principal Component Analysis), nous gardons la plupart des informations qui nous intéressent au sein des embeddings tout en éliminant l'information inutile et réduisant grandement le coût des calculs, permettant un meilleur usage des embeddings à large échelle. Nous avons donc choisi de réduire les vecteurs à 512 dimensions par un *IncrementalPCA*, qui apprend sur les 50 000 premières protéines et identifie ainsi les composants principaux, c'est-à-dire les axes de variation les plus importants. Chaque vecteur sera ensuite projeté dans un espace de 512 dimensions. Les résultats montrent que 98,20% de l'information est conservée avec seulement 3% des dimensions originales, et que 68% de l'information est résumée par seulement les 10 premières composantes (axes du PCA). Ce résultat confirme bien notre première intuition : le nombre important de dimensions des embeddings de ProteinBERT est utilisé pour couvrir un large panel de protéines, mais qu'au sein d'une même famille la plupart des dimensions ont des valeurs identiques. Nous avons ainsi éliminé le bruit et grandement réduit le temps de calcul et la place nécessaire pour la suite.

De plus, toute protéine trop courte ou bien présentant une anomalie est remplacée par un vecteur nul.

15.1.4 Résultats de ProteinBERT

Nous avons choisi d'utiliser ProteinBERT dans ce projet car sa création d'embeddings est directement liée aux fonctions/annotations des protéines, étant donné qu'elles ont constitué une grande part de son entraînement. Le but de notre projet étant la prédiction de fonctions en se basant sur les clusters formés par nos modèles grâce au graphe BLAST et aux scores de similarité cosinus des embeddings, le fait que ces derniers seront plus élevés si deux protéines partagent la même fonction pour ProteinBERT fait qu'elles se retrouveront dans le même cluster. Nous obtiendrons ainsi des clusters avec une bonne homogénéité et peu de protéines incompatibles, ce qui est exactement le résultat attendu.

Le fichier de résultats contient les embeddings des protéines, mais certaines d'entre elles manquent.

Données du fichier proteinBERT - essai 1 :

- Taille du fichier : 13,41 Go
- Nombre total de protéines : 7 426 058
- Nombre de protéines manquantes : 2 422 104
- Pourcentage de protéines manquantes : 32.62 %

Pourquoi certaines protéines manquent-elles à l'appel ?

D'après notre code de création des embeddings utilisant ProteinBERT, si le modèle ou ses fonctions renvoient une erreur, alors nous passons à la protéine suivante. C'est donc ce qui est arrivé à l'ensemble de ces protéines manquantes.

Après analyse des séquences des protéines manquantes, nous avons réalisé qu’elles avaient toutes un point commun : l’**acide aminé X**. En effet, le X représente un acide aminé indéterminé. Bien que dans l’article de ProteinBERT les X soient mentionnés comme une protéine acceptée, en réalité, elles soulèvent une erreur liée à l’encodeur de la librairie de ProteinBERT qui bloque le passage de ces protéines.

Que faire quant à ces protéines manquantes ?

En l’état, ProteinBERT et ses fonctions n’acceptent pas les X. De plus, étant donné qu’il correspond à un acide aminé manquant, nous ne pouvons pas remplacer cette lettre par un acide aminé proche. De même, nous pourrions supprimer ces X, mais cela décalerait la position des acides aminés des protéines ; or c’est l’élément principal qui permet à ProteinBERT de générer ses embeddings. Il vaut donc mieux laisser de côté ces protéines et se concentrer sur les embeddings que nous possédons déjà et qui sont corrects.

Néanmoins, certaines sont passées à la trappe uniquement car une protéine de leur batch provoquait une erreur. Nous devons donc sauver ces protéines.

Pour ce faire, nous avons retiré toutes les protéines comportant des X avant de recalculer les embeddings proprement.

Données des protéines .fasta :

- **Nombre total de protéines** : 7 426 058
- **Nombre de protéines contenant des X** : 1 708 985 (23 %)
- **Nombre de protéines contenant moins de 5 acides aminés** : 9

Données du fichier .csv sans les protéines contenant des X et celles faisant strictement moins de cinq acides aminés :

- **Nombre de protéines contenant plus de 512 acides aminés** : 805 032
- **Nombre de protéines contenant plus de 1024 acides aminés** : 135 195

Données du fichier proteinBERT - essai 2 :

- **Taille du fichier** : 12 Go
- **Nombre total de protéines** : 5 717 064 (77 % du fichier originel)

Toutes les protéines ne contenant pas de X et ayant au moins cinq acides aminés sont à présent liées à un embedding !

15.2 Graphe de similarité BLAST

15.2.1 Traitement du graphe BLAST

Choix du poids des arêtes

Utilisation de 3 métriques :

- pident
- length
- qlen et slen

La formule choisie est :

$$\text{weight} = \frac{\text{pident}}{100} \times \underbrace{\frac{\text{length}}{\min(\text{qlen}, \text{slen})}}_{\text{coverage}}$$

Elle assure :

- valeur entre 0 et 1
- indépendance de la longueur brute
- interprétabilité biologique

Suppression des arêtes dont le poids est en dessous d'un seuil

Pour le moment, le choix du seuil s'est porté sur 0.1 pour construire le graphe. Les arêtes avec un poids inférieur n'ont pas été prises en compte afin d'alléger le graphe et réduire le bruit ainsi que les poids non significatifs.

Nous pouvons envisager un seuil plus élevé afin de réduire encore ces aspects.

Graphe obtenu

- 48 808 941 arêtes (sur 89 280 577 lignes dans *diamond alignments.filter.tsv*)
- stockées dans **graph edges.tsv** avec chaque ligne représentant une arête de la forme : **sequence-proteine1 sequence-proteine2 poids-arete**

15.2.2 Graphe BLAST complet

Propriétés

- Nombre de sommets : 4 930 344 ;
- Nombre d'arêtes : 48 808 941 ;
- Graphe **non connexe** ;
- **Nombre de composantes** : 139 391 ;
- **Nœuds isolés** : 75 550 ;
- **Composantes (>1 nœud)** : 63 841.

On constate qu'il y a plus de nœuds isolés que de composantes. Cela signifie que le graphe BLAST n'a pas pu trouver de similarité pour de nombreuses protéines.

Regardons les statistiques des composantes.

Statistique	Valeur
Min	2
Q1	3
Médiane	5
Q3	13
Max	45892840
Moyenne	763.36
Écart-type	181631.67

TABLE 5 – Distribution des composantes (>1 nœud) BLAST

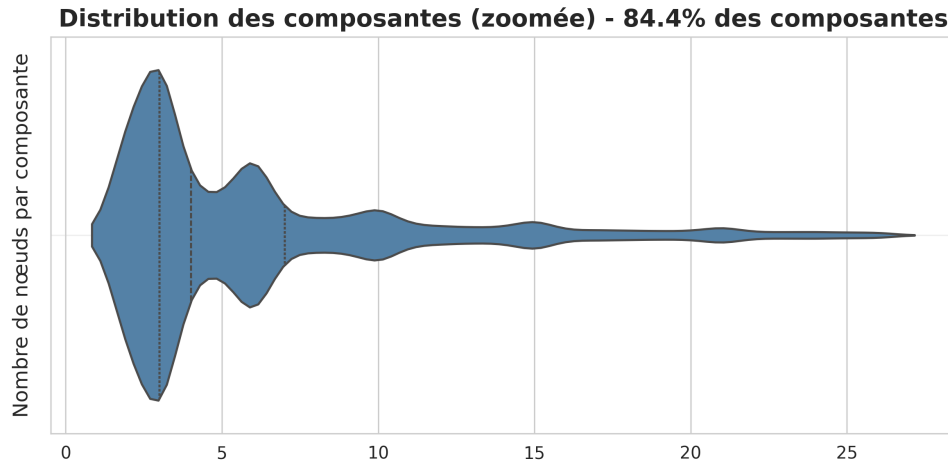


FIGURE 13 – Distribution des composantes ($1 < \text{nombre de nœuds} \leq 2 \times Q3$) BLAST

La plus grande part des composantes a une taille (nombre de nœuds) proche de 3. Nous avons donc essentiellement des petites composantes.

On remarque qu'il y a une **composante principale** bien plus grande que toutes les autres.

- Nombre de sommets : 4 210 972 (85,4% des nœuds du graphe BLAST) ;
- Nombre d'arêtes : 45 892 840 (93% des arêtes du graphe BLAST) ;
- Graphe **connexe** ;
- Densité : 0,000005.

Pour les algorithmes de clustering requérant un graphe connexe, on peut considérer que les composantes forment déjà des clusters et calculer leurs métriques d'évaluation. Pour celles dont les résultats ne sont pas suffisamment bon, il faudra appliquer un algorithme de clustering pour la diviser en plusieurs clusters et recalculer les scores. Ce processus est à répéter jusqu'à ce que les résultats soient satisfaisants.

15.2.3 Graphe BLAST réduit

Le graphe BLAST réduit ne comprend que les protéines qui se trouvent également dans les embeddings. Il a été utilisé pour la comparaison avec la combinaison linéaire pondérée de BLAST et des embeddings.

Propriétés

- **Nombre de sommets** : 3 776 093 (76,6% de BLAST complet) ;
- **Nombre d'arêtes** : 32 407 576 (66,4% de BLAST complet) ;
- Graphe **non connexe** ;
- **Nombre de composantes** : 112 511 (80,4% de BLAST complet) ;
- **Nœuds isolés** : 61 682 (81,64% de BLAST complet) ;
- **Composantes (>1 nœud)** : 51 576 (80,8% de BLAST complet) ;
- **Composante principale** : 3 175 135 nœuds, 29 998 174 arêtes ;
- **Densité** : 0.000006.

Statistique	Valeur
Min	2
Q1	3
Médiane	6
Q3	15
Max	29998173
Moyenne	627.15
Écart-type	132089.00

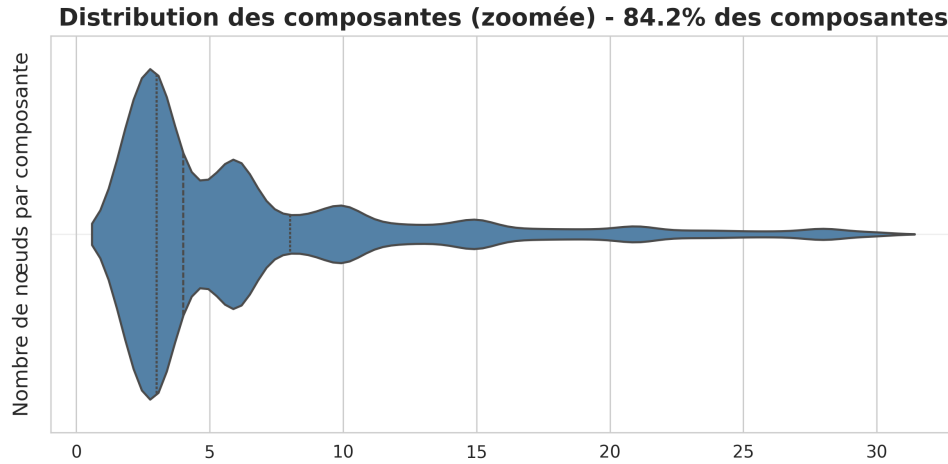


FIGURE 14 – Distribution des composantes ($1 < \text{nombre de nœuds} \leq 2 \times Q3$) BLAST réduit

15.3 Graphe combinant BLAST et les embeddings

15.3.1 Calcul de la similarité cosinus entre les embeddings

Afin d'optimiser les calculs, nous n'avons calculé la similarité cosinus qu'entre les protéines qui sont déjà en relation dans le graphe BLAST, puisque nous souhaitons réaliser une combinaison linéaire pondérée entre la similarité cosinus obtenue avec les embeddings et les poids du graphe BLAST.

Qu'est-ce que la similarité cosinus entre deux embeddings ?

Les embeddings sont des vecteurs. Il suffit de prendre la formule classique du produit scalaire entre deux vecteurs $\vec{v}_1$ et $\vec{v}_2$, avec α l'angle entre ces deux vecteurs :

$$\langle \vec{v}_1, \vec{v}_2 \rangle = \cos(\alpha) \times \|\vec{v}_1\| \times \|\vec{v}_2\|$$

Pour obtenir la formule que nous souhaitons, il faut isoler le cosinus :

$$\text{Similarité cosinus} = \cos(\alpha) = \frac{\langle \vec{v}_1, \vec{v}_2 \rangle}{\|\vec{v}_1\| \times \|\vec{v}_2\|}$$

Comment interpréter les résultats ?

Valeurs prises par la similarité cosinus entre deux vecteurs et leur signification :

- 1 ($\alpha = 0^\circ$) : les vecteurs sont parfaitement alignés. Les protéines ont des fonctions identiques pour le modèle.
- 0 ($\alpha = 90^\circ$) : les vecteurs sont orthogonaux. Les protéines ne sont pas similaires.

- -1 ($\alpha = 180^\circ$) : les vecteurs sont opposés.

Statistiques

- **Nombre d’Ids uniques dans les embeddings** : 5 717 064
- **Nombre d’Ids uniques dans BLAST** : 4 930 344
- **Nombre d’Ids présents dans BLAST mais pas dans les embeddings** : 1 088 783 (22,08 %)

Seuil

Après avoir calculé la similarité cosinus, un seuil de 0.1 leur a été appliqué, le même que pour BLAST.

15.3.2 Fusion du graphe BLAST et de la similarité cosinus des embeddings

Afin de fusionner nos deux graphes, nous opérons par combinaison linéaire pondérée des poids des arêtes.

$$\text{Score final} = \alpha \times S_{\text{BLAST}} + (1 - \alpha) \times S_{\text{embed}} \text{ avec } \alpha \in [0, 1]$$

Afin de trouver la valeur optimale pour α , nous avons construit sept graphes différents en prenant : $\alpha \in \{0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0\}$.

De nombreuses protéines de BLAST sont absents des embeddings. Nous avons donc choisi de supprimer les arêtes liées à l’une de ces protéines du graphe final.

Statistiques de chaque graphe

- **Nombre de sommets** : 3 776 093 ;
- **Nombre d’arêtes** : 32 407 576 ;
- **Nombre de composantes** : 113 258 ;
- **Nœuds isolés** : 61 682 ;
- **Composantes (>1 nœud)** : 51 576 ;
- **Composante principale** : 3 175 135 nœuds, 29 998 174 arêtes ;
- **Densité** : 0,000006.

On constate qu’il y a plus de nœuds isolés que de composantes. Cela signifie que le graphe BLAST n’a pas pu trouver de similarité pour de nombreuses protéines.

Statistique	Valeur
Min	2
Q1	3
Médiane	6
Q3	15
Max	29998174
Moyenne	627.15
Écart-type	132089.01

TABLE 6 – Distribution des composantes (>1 nœud) Fusion BLAST et embeddings

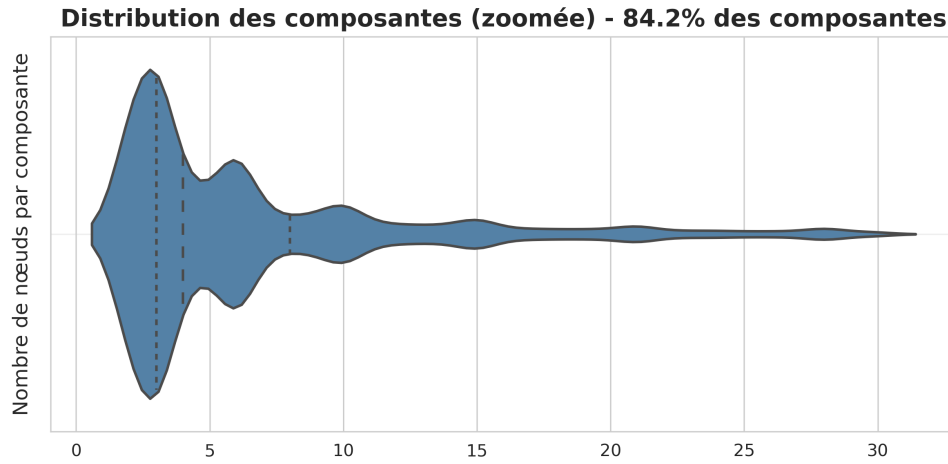


FIGURE 15 – Distribution des composantes ($1 < \text{nombre de nœuds} \leq 2 \times Q3$) BLAST

La plupart des composantes ont une taille (nombre de nœuds) proche de 3. Nous avons donc essentiellement de petites composantes.

15.3.3 Implémentation des métriques d'évaluation

Dans ce projet, nous considérons deux métriques classique pour le clustering de graphe, qui nous permettent de juger l'homogénéité de la partition et la séparabilité de ses clusters.

- **Co-distance** : somme des distances intra-cluster ;
- **Coupe normalisée** : divise la valeur de la coupe (somme des poids des arêtes reliant un cluster au reste de la partition) par le volume total du cluster (somme de tous les poids de ses arêtes).

Pour chaque cluster, nous calculons ces métriques d'évaluation, puis nous regardons leurs statistiques sur la partition globale. Si un cluster n'est pas suffisamment homogène, il pourra alors être nécessaire de refaire du clustering pour le séparer, jusqu'à obtenir des clusters satisfaisants.

15.4 Clustering avec Leiden et MCL

15.4.1 Méthode

Leiden

L'algorithme de Leiden est un algorithme de clustering de graphes basé sur l'optimisation de la modularité. Il est particulièrement efficace pour les graphes de grande taille.

Principe de l'algorithme Il opère sur un graphe avec un seul hyperparamètre :

- **Résolution** : ajuste la sensibilité du découpage.

Choix des paramètres Le paramètre de résolution γ contrôle la granularité du clustering :

- $\gamma < 1.0$: favorise de plus gros clusters au détriment de leur homogénéité ;
- $\gamma > 1.0$: favorise de petits clusters très denses.

Dans notre étude, nous avons fait varier ce paramètre avec les valeurs $\gamma \in \{0.1, 0.5, 1.0, 2.0, 5.0\}$ afin d'explorer différents niveaux de granularité.

Choix des bibliothèques utilisées Nous avons choisi les bibliothèques *leidenalg* et *igraph* pour cette implémentation. La première permet d'exécuter l'algorithme de Leiden sur le graphe passé en paramètre, et la seconde permet de créer ce même graphe. Pour *leidenalg*, le paramètre `partition_type` donnant la partition utilisée a reçu l'implémentation `RBConfigurationVertexPartition`. Cette fonction permet d'ajuster la valeur du paramètre de résolution γ à notre guise, là où d'autres partitions le fixent, de sorte qu'on puisse trouver la valeur idéale pour notre graphe. De plus, l'algorithme est ainsi stable sur les grands réseaux, ce qui nous est indispensable. Enfin, nous l'avons préféré à `CPMVertexPartition`, car RB est plus adapté aux graphes où le poids des arêtes varie beaucoup, ce qui est notre cas.

MCL

L'algorithme de clustering de Markov (MCL) est un algorithme de clustering de graphes basé sur la simulation de flux au sein du réseau. Il repose sur l'idée que des marches aléatoires ont tendance à rester confinées dans des régions fortement connectées du graphe, ce qui permet d'identifier naturellement des communautés de nœuds.

Principe de l'algorithme MCL opère sur une matrice de transition dérivée de la matrice d'adjacence du graphe. Il alterne deux opérations principales :

- **Expansion** : correspond à une multiplication matricielle simulant des marches aléatoires de longueur supérieure à 1. Cette étape permet de diffuser le flux à travers le graphe.
- **Inflation** : consiste à élever chaque coefficient de la matrice à une puissance r , puis à renormaliser les colonnes. Cette opération renforce les connexions fortes et atténue les connexions faibles.

L'alternance de ces deux étapes permet de faire émerger progressivement des structures de clusters en concentrant le flux dans des sous-graphes denses.

Choix des paramètres Le paramètre principal de MCL est le paramètre d'inflation r , qui contrôle directement la granularité du clustering :

- des valeurs faibles de r favorisent la formation de clusters plus larges, mais potentiellement moins homogènes ;
- des valeurs élevées de r conduisent à des clusters plus petits et plus denses.

De manière analogue au paramètre de résolution γ dans l'algorithme de Leiden, le paramètre r permet donc d'ajuster le niveau de détail du clustering.

Dans notre étude, nous avons fait varier ce paramètre avec les valeurs $r \in \{1.5, 2.0, 3.0\}$ afin d'explorer différents niveaux de granularité.

D'après la documentation de l'algorithme MCL utilisé, le paramètre d'inflation pourrait varier de 1.1 à 10.0. Cependant, la gestion de ce paramètre dépend également du type de graphe considéré. Le nôtre est très sparse, avec une densité très faible. Le paramètre d'inflation ne peut donc pas descendre en-dessous de 1.5, ou bien l'algorithme sera très long à s'exécuter, voire ne convergera jamais. De plus, la littérature en bioinformatique utilise typiquement 1.5 pour obtenir de gros clusters tout en garantissant la convergence.

Paramètre d'expansion Le paramètre d'expansion a été fixé à 2, qui correspond à la valeur standard dans la littérature. Ce choix permet de simuler des marches aléatoires de longueur suffisante pour capturer la structure du graphe, tout en limitant le coût computationnel.

Nous avons également testé ponctuellement une valeur d'expansion différente ($e = 3$) afin de vérifier son influence, mais celle-ci n'apporte pas d'amélioration significative aux résultats. Par conséquent, nous avons conservé $e = 2$ pour l'ensemble des expériences.

Application aux graphes Contrairement à certains algorithmes de clustering, MCL peut être appliqué directement à des graphes non connexes. En effet, le flux simulé ne peut pas traverser les composantes connexes, ce qui implique que chaque composante est traitée indépendamment sans nécessiter de prétraitement spécifique.

Cependant, afin de pouvoir comparer les résultats de Leiden et MCL, nous leur avons fourni à tous deux la composante principale connexe de nos graphes obtenus par combinaison linéaire pondérée entre BLAST et les embeddings.

Choix des bibliothèques utilisées Nous avons choisi la bibliothèque `mcl` pour cette implémentation. Nous l'avons sélectionnée en grande partie car notre graphe est massif, ce qui demande des algorithmes optimisés. Un MCL naïf saturerait la RAM instantanément à cause de l'étape d'Expansion. Cette bibliothèque est idéale car elle intègre des algorithmes d'élagage dynamique hautement optimisés. À chaque étape d'Inflation, les probabilités presque nulles sont supprimées de la mémoire. La matrice reste creuse (sparse), ce qui permet à l'algorithme de tourner sur des millions de lignes sans faire exploser le serveur de calcul.

15.4.2 Résultats

Graphe combinant BLAST et les embeddings pour différentes valeurs de α

Rappel des α sélectionnés : $\alpha \in \{0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0\}$.

Nous avons appliqué les deux méthodes sur la composante principale de chacun des graphes.

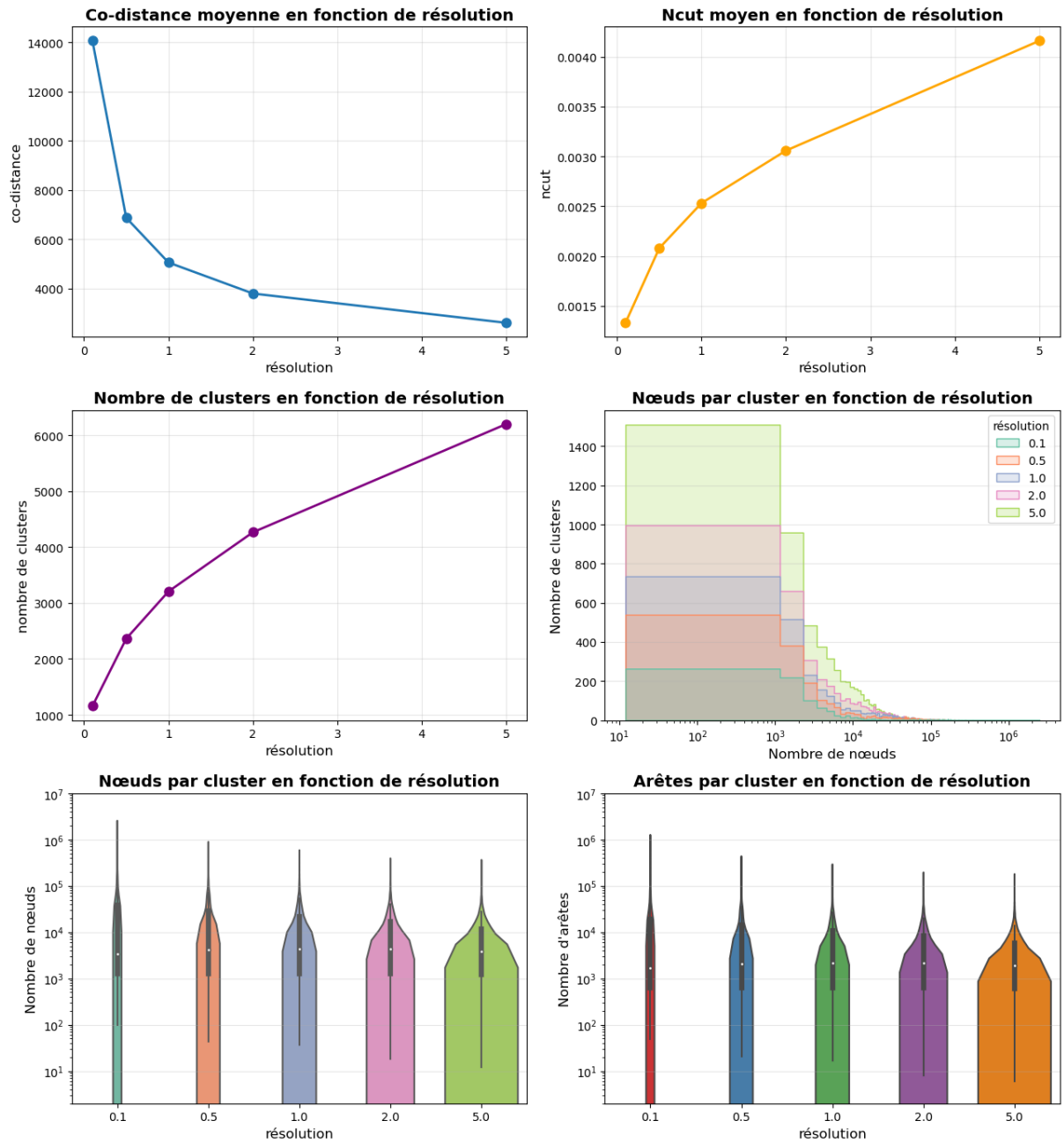


FIGURE 16 – Évolution des métriques sur Leiden en fonction de gamma - graphe combinant BLAST et les embeddings - $\alpha = 0.5$

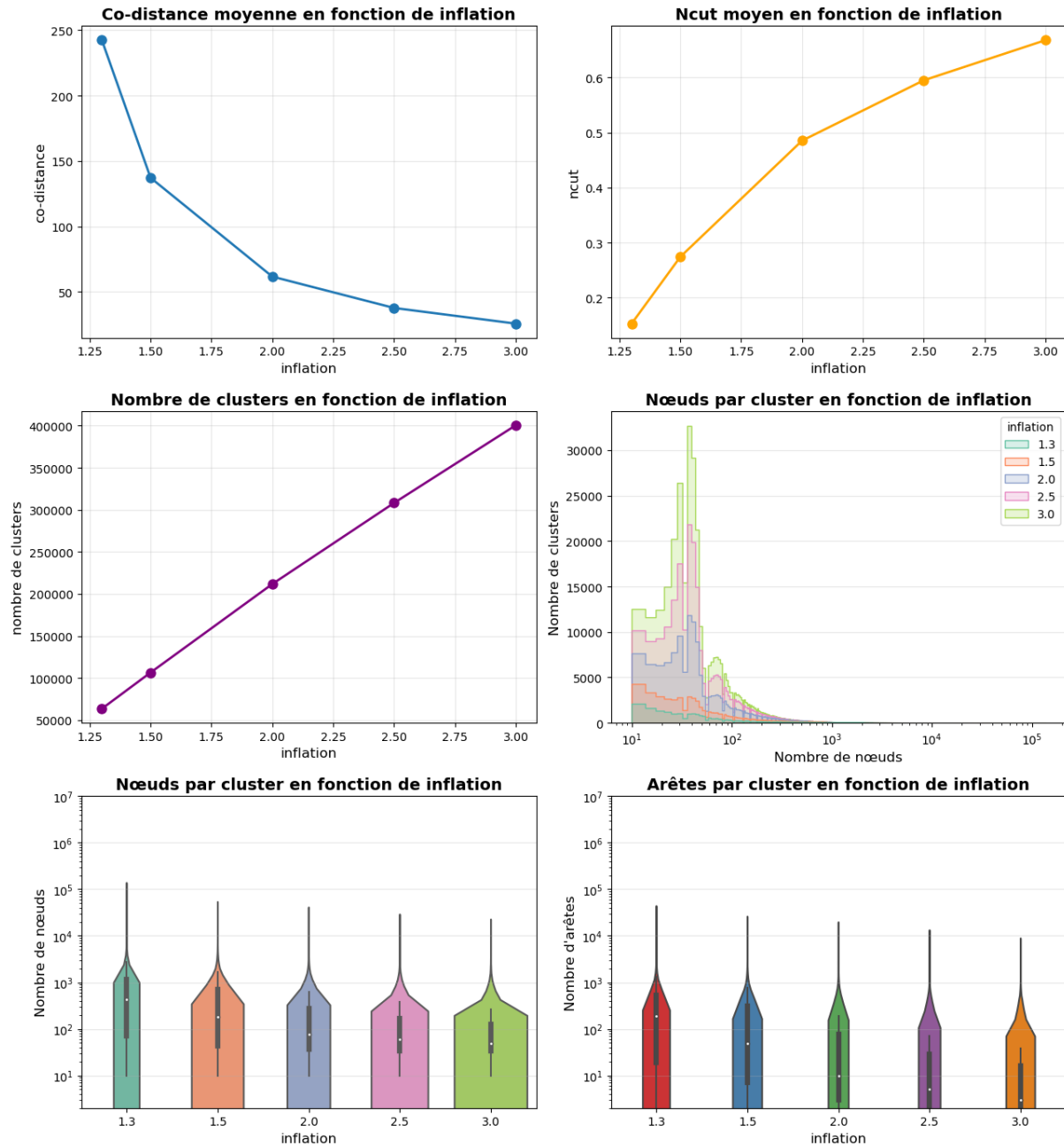


FIGURE 17 – Évolution des métriques sur MCL en fonction de l’inflation - graphe combinant BLAST et les embeddings - $\alpha = 0.5$

Note : Les échelles diffèrent pour souligner la granularité spécifique de chaque méthode.

Différences entre les deux modèles Les graphes des deux modèles sont très différents. En effet, on constate que les partitions produites par le clustering ont des échelles très différentes. Dans le cas de Leiden, granularité est beaucoup plus faible que pour MCL, qui lui va tendre à fragmenter le graphe en petits clusters. Leiden tolère des liens plus faibles à l’intérieur d’une communauté tant que cela améliore le score global du réseau. Cela est dû au fait qu’il cherche à maximiser la modularité globale. MCL, lui, a tendance à faire beaucoup de petits clusters même avec un paramètre d’inflation assez bas.

Similarités entre les deux modèles Les courbes suivent les mêmes tendances entre les deux modèles. La Ncut moyenne de MCL est bien plus élevée que celle de Leiden étant

donné que l'algorithme a tendance à faire de nombreux petits clusters donc à couper beaucoup plus d'arêtes.

Conclusion sur le choix des paramètres des modèles En utilisant la méthode du coude, on observe qu'un paramètre de résolution idéal serait vers 1.0, et 2.0 pour l'inflation. Ce sont les deux coudes les plus apparents pour tous les graphes.

Impact du alpha Les graphes correspondant aux embeddings et à BLAST ont été placés dans l'annexe (??). La granularité des clusters obtenus par les embeddings seuls est très différente de celle des autres. En effet, la partition comporte de nombreux clusters avec un nombre moindre de nœuds. On peut l'interpréter comme étant le fait que BLAST identifie les plus faibles liaisons entre les protéines, ce qui fait que Leiden, très sensible au moindre changement de poids, les regroupe dans les mêmes clusters, tandis que les embeddings ignorent ces faibles liaisons. Les deux graphes se complètent avec la combinaison linéaire pondérée, les embeddings prenant principalement en compte la proximité fonctionnelle des protéines, donc ayant une vision plus globale, tandis que BLAST regarde plutôt une homologie de séquence.

Comparaison des clusters obtenus entre le graphe BLAST et le graphe combinant BLAST et les embeddings

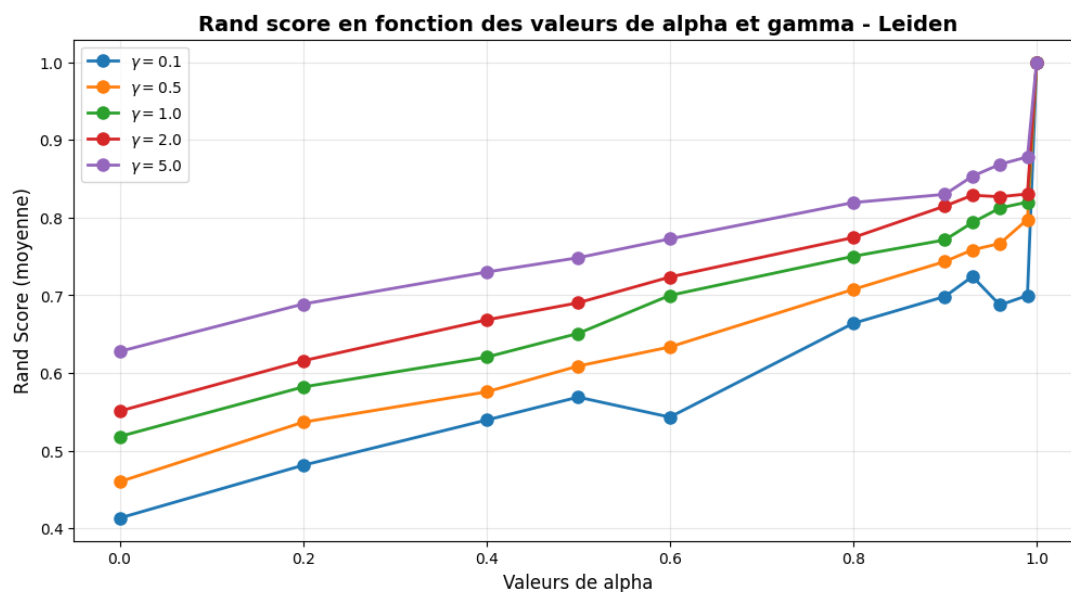


FIGURE 18 – Leiden - Rand score entre BLAST et BLAST & Embeddings - évolution pour chaque γ en fonction des valeurs de α

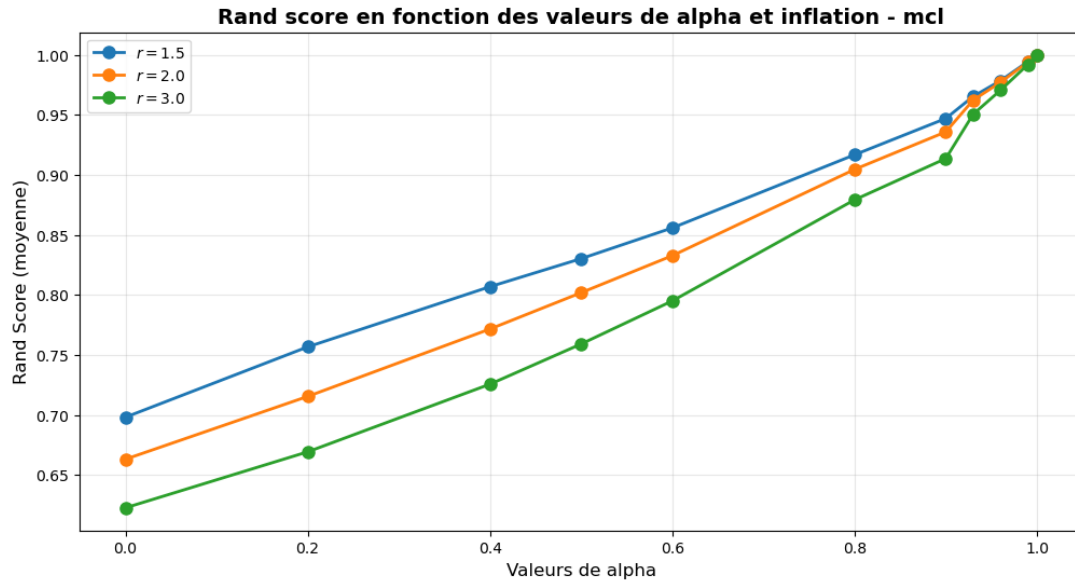


FIGURE 19 – MCL - Rand score entre BLAST et BLAST & Embeddings - évolution pour chaque r en fonction des valeurs de α

On constate l'Index de Rand augmente de façon quasi linéaire avec α . Cela indique que **les embeddings et BLAST ne capturent pas les mêmes signaux biologiques**, puisque leurs scores de similarité sont différents. Si les poids étaient redondants, on aurait un Index de Rand bien plus élevé dès $\alpha = 0,5$.

Pour **Leiden**, on observe un pic de l'Index de Rand entre les valeurs 0,99 et 1,0 de α . Ainsi donc, un très léger changement dans les poids résulte dans des partitions très différentes avec Leiden. En effet, c'est un algorithme de détection de communautés très sensible : il suffit que quelques arêtes changent de poids pour qu'une fusion ou une scission se produisent en cascade. De plus, avec un α de 0,0, soit uniquement des embeddings, l'Index de Rand est en-dessous de 0,5 (aléatoire) pour un γ inférieur à 0,5, et au-dessus sinon.

Pour **MCL**, les valeurs de l'Index de Rand sont globalement supérieures à celles de Leiden, minimisées par 0,6. Cela est dû au fait que les clusters produits sont bien plus petits et plus nombreux, donc seules les arêtes de poids fort restent. Et ces fortes ressemblances sont trouvées à la fois par BLAST et par les embeddings.

De plus, on constate que les courbes d'inflation sont parallèles, ce qui suggère que MCL est robuste. Quelle que soit l'inflation, l'impact des embeddings reste le même.

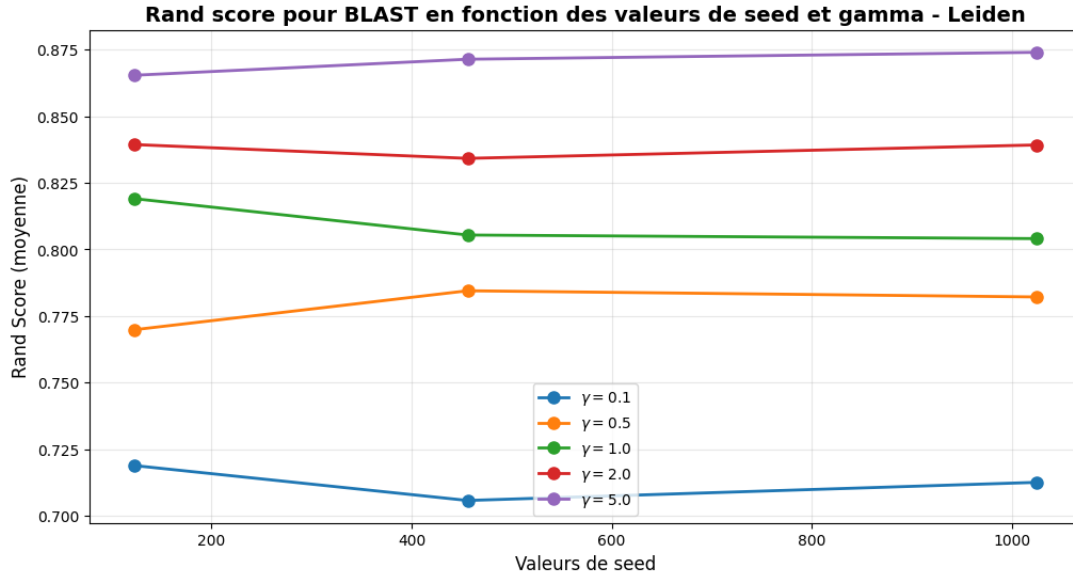


FIGURE 20 – Rand score entre BLAST seed 42 et trois autres seeds - Leiden

Les variations de graine donnent des clusters similaires avec Leiden, le choix de la graine n'a donc que peu d'importance.

Vérification de la complexité

Vérifions les complexités respectives de nos deux algorithmes en relevant leurs temps sur les 20 plus grandes composantes du graphe BLAST. La taille de ces composantes varie de 414 à 3 175 135 nœuds.

La complexité théorique de l'algorithme de **Leiden** est de $O(n \log(n))$, tandis que la complexité donnée en pratique est de $O(n)$.

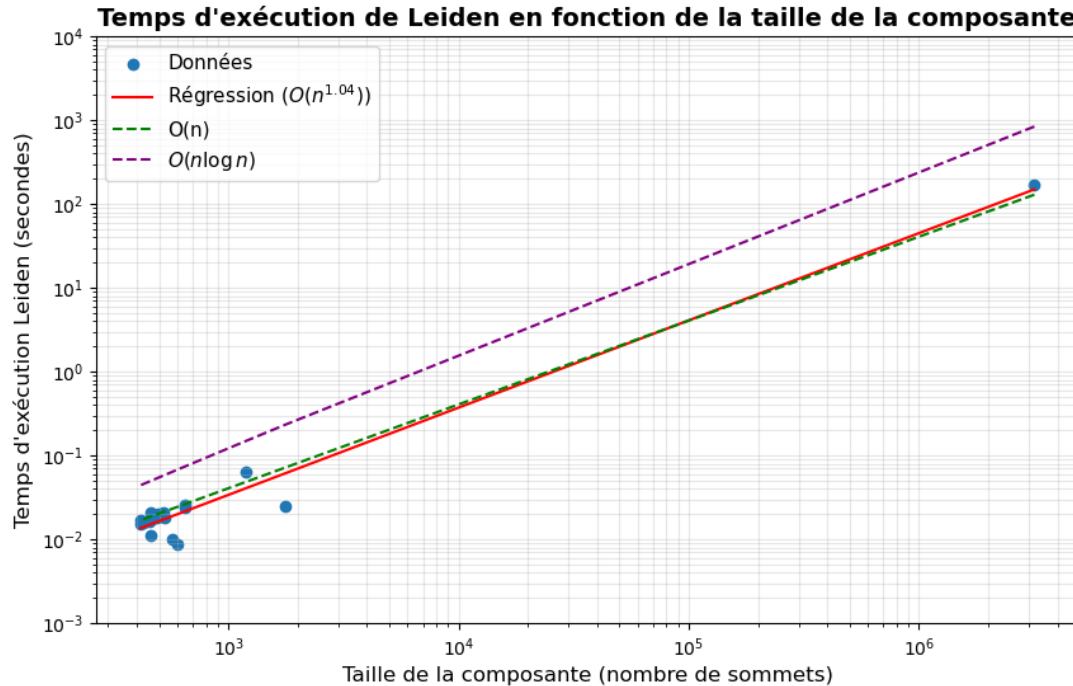


FIGURE 21 – Complexité de l'algorithme de Leiden

La complexité trouvée est quasi-linéaire ($n^{1.04}$).

On constate que le temps mesuré augmente avec la taille des composantes, mais est tout d'abord inférieur aux deux courbes de complexité, puis passe au-dessus de celle de la complexité en pratique. Il est proche de $O(n)$ pour de petites composantes, mais tend vers $O(n \log(n))$ pour de plus grandes composantes. Ce résultat était attendu pour des matrices creuses.

La complexité théorique de l'algorithme de **MCL** est de $O(n^3)$, tandis que la complexité donnée en pratique est de $O(n)$.

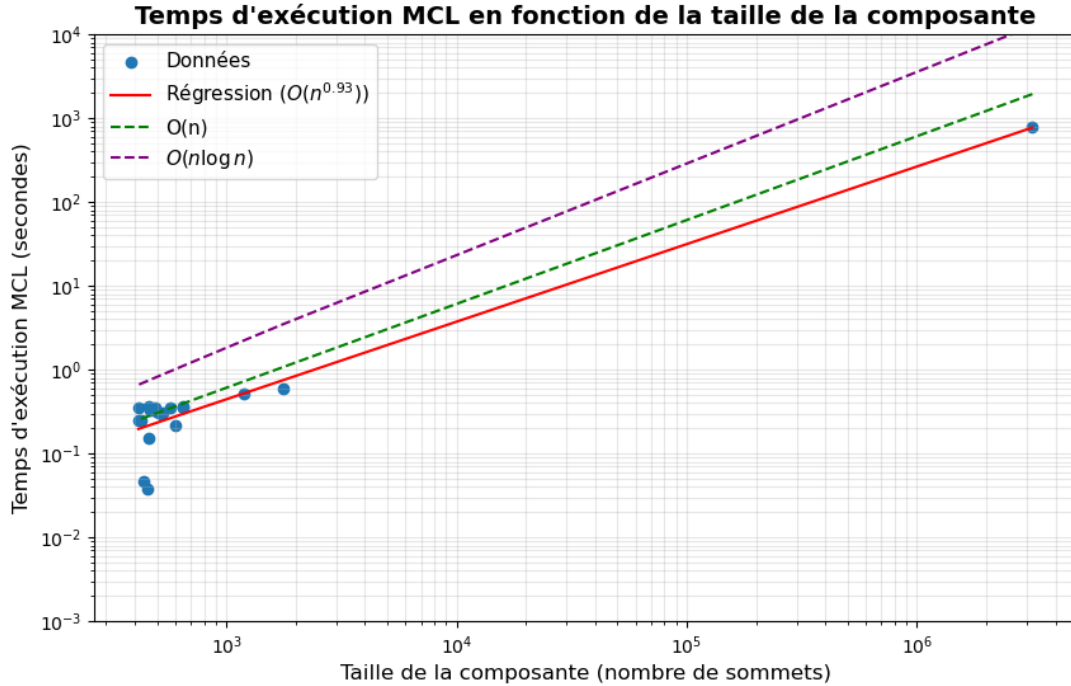


FIGURE 22 – Complexité de l'algorithme MCL

La complexité trouvée est quasi-linéaire ($n^{0.93}$).

On observe que le temps d'exécution augmente avec la taille des composantes, mais reste très inférieur à la borne théorique $O(n^3)$ sur ces composantes. Cela s'explique par le fait que les composantes testées sont relativement petites et que le graphe est creux. En pratique, MCL bénéficie donc de la faible densité du graphe, mais son coût reste plus sensible à la taille des composantes que celui de Leiden, car il repose sur des opérations matricielles répétées lors des étapes d'expansion et d'inflation.

Cette expérience confirme que l'application de MCL sur de très grandes composantes doit être réalisée avec précaution, ce qui justifie le choix d'un nombre limité de paramètres testés et l'utilisation d'une valeur standard pour le paramètre d'expansion.

Néanmoins, il est surprenant de constater cette inversion de performance dans les tests. Cela s'explique tout d'abord par notre choix de modèles, le script MCL appelle un binaire compilé en langage C, très optimisé. De plus, il élague les valeurs faibles des matrices, et comme dit plus haut, notre graphe est très peu dense, donc la vitesse de l'algorithme augmente drastiquement. De son côté, l'utilisation de `igraph` en python pour Leiden peut ralentir le processus sur des graphes massifs.

15.5 Évaluation des partitions et de la propagation à partir des annotations PFAM

Les annotations PFAM sont utilisées à deux niveaux dans notre analyse. Dans un premier temps, elles permettent d'évaluer la cohérence biologique des clusters obtenus, indépendamment de toute prédiction. Dans un second temps, elles permettent d'évaluer la qualité des annotations propagées, en masquant temporairement une partie des annotations connues.

Ces deux niveaux d'évaluation ne mesurent pas exactement la même chose. L'analyse des clusters cherche à déterminer si une partition est exploitable biologiquement, c'est-à-dire si les protéines regroupées partagent des annotations compatibles. L'évaluation de propagation mesure quant à elle la capacité réelle d'une méthode à prédire des annotations masquées.

15.5.1 Évaluation de l'exploitabilité des clusters

Les premières analyses fondées uniquement sur le score moyen de Jaccard se sont révélées peu discriminantes. En effet, un score moyen élevé peut être obtenu sur un petit nombre de protéines annotées, ou sur des clusters de très petite taille. Nous avons donc défini une notion de cluster exploitable afin d'éviter les interprétations fondées sur des cas trop peu informatifs.

Un cluster est considéré comme exploitable s'il satisfait les trois conditions suivantes :

- il contient au moins 5 protéines ;
- au moins 30% de ses protéines possèdent une annotation PFAM ;
- il contient au moins 10 paires évaluables.

Le seuil de 30% d'annotations est choisi car il est proche de la proportion globale de protéines annotées PFAM dans l'ensemble du jeu de données, égale à 30.80%. Il permet donc de sélectionner des clusters suffisamment informatifs sans être trop restrictif.

Proportion de clusters exploitables La figure 23 présente la proportion de clusters exploitables en fonction de la valeur de α .

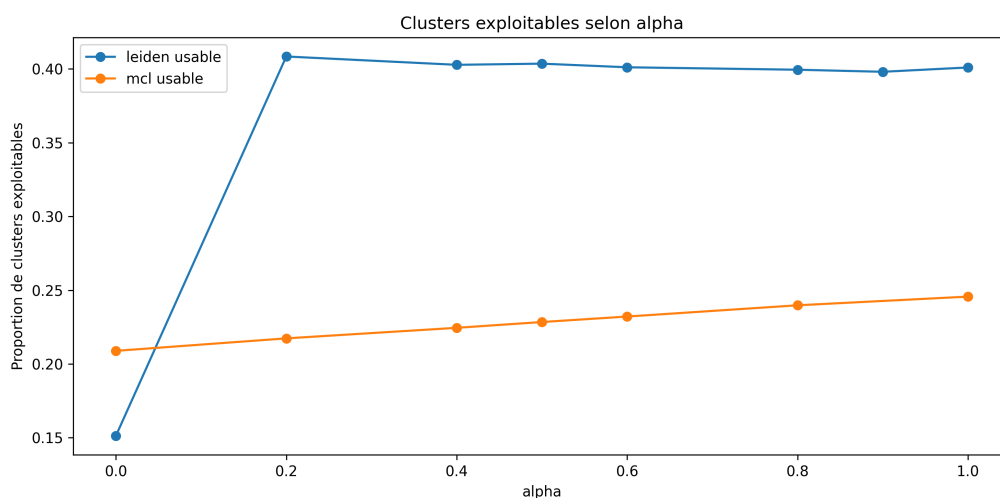


FIGURE 23 – Proportion de clusters exploitables en fonction de α pour Leiden et MCL.

Pour Leiden, le cas $\alpha = 0$, correspondant aux embeddings seuls, donne une faible proportion de clusters exploitables. Dès que l'on réintroduit une contribution de BLAST dans

la combinaison, la proportion de clusters exploitables augmente fortement et se stabilise autour de 40%. Cela montre que les embeddings seuls ne suffisent pas à produire des clusters facilement interprétables à partir des annotations PFAM.

Pour MCL, la proportion de clusters exploitables varie moins avec α . Elle reste globalement plus faible et plus stable, ce qui s'explique par la forte fragmentation produite par cet algorithme. MCL génère un très grand nombre de petits clusters, dont beaucoup ne contiennent pas assez de paires annotées pour être réellement informatifs.

Clusters sans incompatibilité Nous avons également étudié les clusters ne contenant aucune paire incompatible parmi les paires évaluables. Un cluster est considéré comme sans incompatibilité si :

$$\text{evaluated_pairs} \geq 10 \quad \text{et} \quad \text{ratio_incompatible} = 0.$$

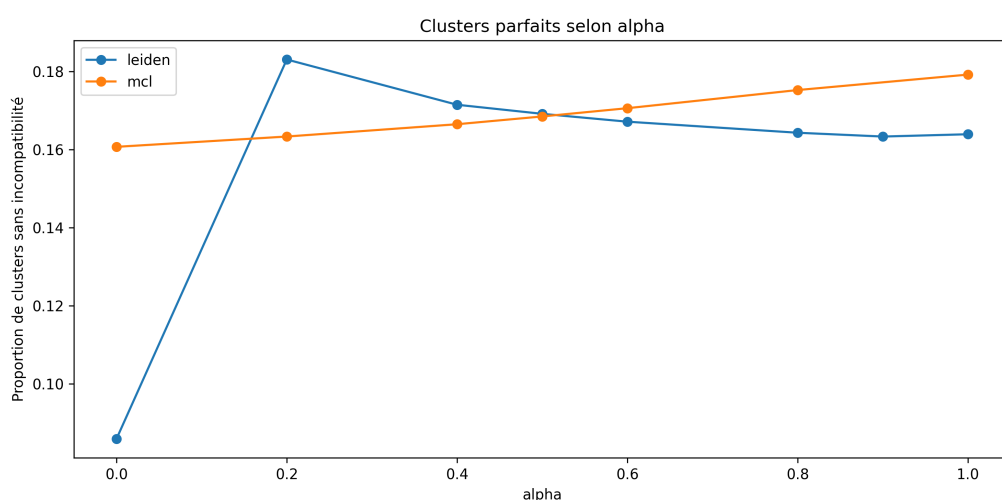


FIGURE 24 – Proportion de clusters sans incompatibilité en fonction de α .

MCL produit davantage de clusters sans incompatibilité que Leiden. Cependant, cette observation doit être interprétée avec prudence : comme MCL fragmente fortement le graphe, il est mécaniquement plus facile d'obtenir de petits clusters parfaitement homogènes. À l'inverse, Leiden produit des clusters plus larges, donc plus difficiles à rendre parfaitement homogènes, mais potentiellement plus utiles pour une propagation d'annotations à grande échelle.

Distribution des tailles de clusters La différence de comportement entre Leiden et MCL apparaît clairement lorsque l'on compare le nombre de clusters et leur taille médiane.

Méthode	α	Nombre de clusters	Taille médiane
Leiden	0.0	96 332	4
Leiden	0.2	20 548	155
Leiden	0.5	17 195	215
Leiden	1.0	15 730	254
MCL	0.0	1 267 982	4
MCL	0.2	1 213 006	4
MCL	0.5	1 137 796	5
MCL	1.0	1 143 342	5

TABLE 7 – Comparaison du nombre de clusters et de leur taille médiane pour Leiden et MCL.

Leiden produit beaucoup moins de clusters, mais ceux-ci sont nettement plus grands lorsque BLAST est intégré à la combinaison. MCL produit au contraire plus d'un million de clusters, avec une taille médiane comprise entre 4 et 5 protéines. Cette fragmentation explique pourquoi MCL obtient souvent de bons scores d'homogénéité locale : les petits clusters ont naturellement moins de chances de contenir des annotations incompatibles.

Ces résultats montrent qu'il ne faut pas interpréter l'homogénéité fonctionnelle indépendamment de la taille des clusters. Un cluster très homogène mais très petit peut être peu utile pour une propagation d'annotations à grande échelle.

Distribution du taux d'incompatibilité La figure 25 présente la distribution du taux de paires incompatibles dans les clusters exploitables.

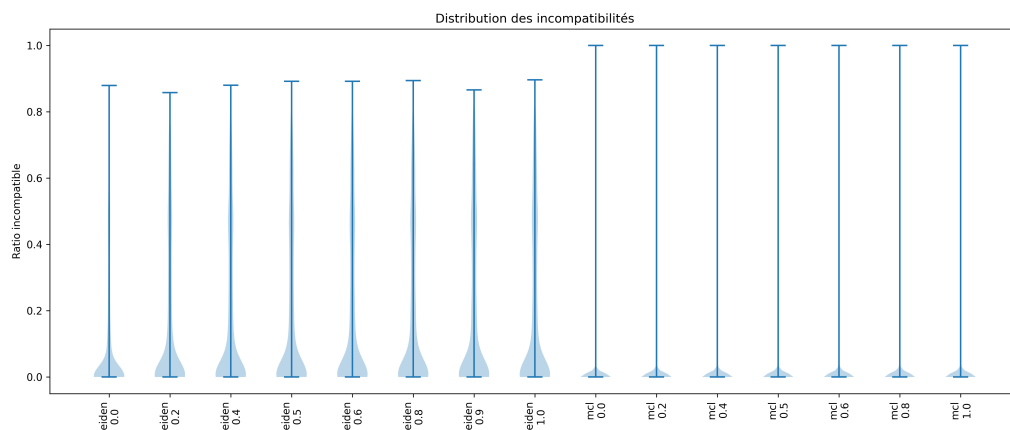


FIGURE 25 – Distribution du taux d'incompatibilité dans les clusters exploitables.

MCL présente une distribution fortement concentrée autour de 0, ce qui confirme que ses clusters exploitables sont souvent très homogènes localement. Leiden présente une distribution plus étalée : ses clusters contiennent davantage de diversité fonctionnelle, mais ils regroupent aussi plus de protéines et sont donc plus informatifs pour une analyse globale.

Ainsi, l'analyse des annotations PFAM met en évidence un compromis important : MCL maximise l'homogénéité locale grâce à une forte fragmentation, tandis que Leiden produit des clusters plus larges, plus riches, mais aussi plus hétérogènes.

15.5.2 Résultats de propagation pour la première approche

Dans la première approche, les poids du graphe sont obtenus par combinaison linéaire entre le score BLAST et la similarité cosinus des embeddings :

$$S_{\text{final}} = \alpha S_{\text{BLAST}} + (1 - \alpha) S_{\text{cos}}.$$

Les partitions obtenues avec Leiden et MCL sont ensuite utilisées pour propager les annotations. L'objectif est d'évaluer si la modification des poids du graphe permet d'améliorer la prédiction d'annotations par rapport à BLAST seul.

La figure 26 présente l'évolution du F1-score micro en fonction de α pour Leiden et MCL.

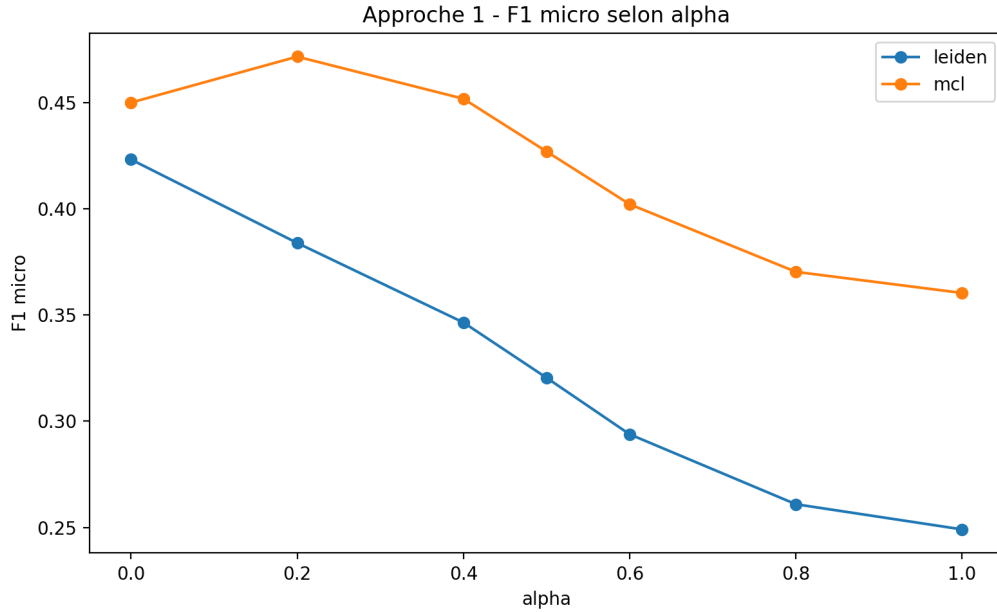


FIGURE 26 – F1-score micro de la propagation pour la première approche en fonction de α .

Les résultats montrent que l'ajout des embeddings améliore les performances de propagation par rapport à BLAST seul. Pour MCL, le meilleur résultat est obtenu avec $\alpha = 0.2$, ce qui signifie que la meilleure configuration conserve une contribution majoritaire des embeddings tout en gardant une part d'information issue de BLAST. Cela indique que les embeddings apportent un signal complémentaire utile, mais que BLAST reste nécessaire pour stabiliser les prédictions.

Configuration	Couverture	Précision micro	Rappel micro	F1 micro
Leiden, $\gamma = 1.0$, $\alpha = 0.0$	39.09%	80.46%	28.72%	42.33%
MCL, $r = 2.0$, $\alpha = 0.2$	36.34%	90.19%	31.94%	47.17%

TABLE 8 – Meilleures configurations de propagation obtenues avec l'approche 1.

Pour Leiden, avec $\gamma = 1.0$, la meilleure configuration de propagation est obtenue pour $\alpha = 0.0$, c'est-à-dire avec le graphe construit uniquement à partir des similarités cosinus des embeddings. Cette configuration atteint une couverture de 39.09%, une précision micro de 80.46%, un rappel micro de 28.72% et un F1 micro de 42.33%.

Pour MCL, avec $r = 2.0$, la meilleure configuration est obtenue pour $\alpha = 0.2$. Elle présente une couverture légèrement plus faible, égale à 36.34%, mais une précision micro

nettement plus élevée, égale à 90.19%. Elle obtient également le meilleur rappel micro, 31.94%, et le meilleur F1 micro, 47.17%.

Ainsi, pour la tâche finale de propagation d’annotations, MCL avec $r = 2.0$ et $\alpha = 0.2$ constitue la meilleure configuration de l’approche 1. Ce résultat nuance le choix de Leiden effectué lors de l’analyse structurale : Leiden produit des clusters plus grands et plus exploitables pour l’analyse globale, mais MCL donne ici les meilleures performances prédictives.

La première approche montre donc que les embeddings ne doivent pas être utilisés seuls, mais qu’ils améliorent la propagation lorsqu’ils sont combinés à BLAST de manière contrôlée.

15.5.3 Résultats de propagation pour la deuxième approche

La deuxième approche obtient une couverture plus élevée que la première approche : elle prédit au moins une annotation pour 42.01% des protéines du jeu de test. Cependant, cette augmentation de couverture s’accompagne d’une baisse importante de précision, qui atteint 75.32%. Le F1-score micro obtenu est de 43.34%.

Méthode	Couverture	Précision micro	Rappel micro	F1 micro
Co-propagation BLAST / embeddings	42.01%	75.32%	30.42%	43.34%

TABLE 9 – Résultats de propagation obtenus avec la deuxième approche.

Ces résultats montrent que la deuxième approche permet d’annoter davantage de protéines, mais au prix d’une augmentation du nombre de faux positifs. Elle est donc intéressante si l’objectif principal est d’obtenir un grand nombre de propositions d’annotations candidates, mais elle est moins satisfaisante si l’on privilégie la fiabilité des prédictions.

En pratique, cette approche montre que BLAST et les embeddings apportent bien des informations complémentaires. Cependant, la propagation alternée peut aussi amplifier certaines erreurs, en particulier lorsque des annotations peu fiables sont transférées d’une partition à l’autre.

15.5.4 Comparaison des deux approches

Le tableau 10 compare les meilleures configurations obtenues pour les deux approches.

Méthode	Couverture	Précision micro	Rappel micro	F1 micro
Approche 1 – Leiden, $\gamma = 1.0$, $\alpha = 0.0$	39.09%	80.46%	28.72%	42.33%
Approche 1 – MCL, $r = 2.0$, $\alpha = 0.2$	36.34%	90.19%	31.94%	47.17%
Approche 2 – co-propagation	42.01%	75.32%	30.42%	43.34%

TABLE 10 – Comparaison finale des meilleures configurations de propagation.

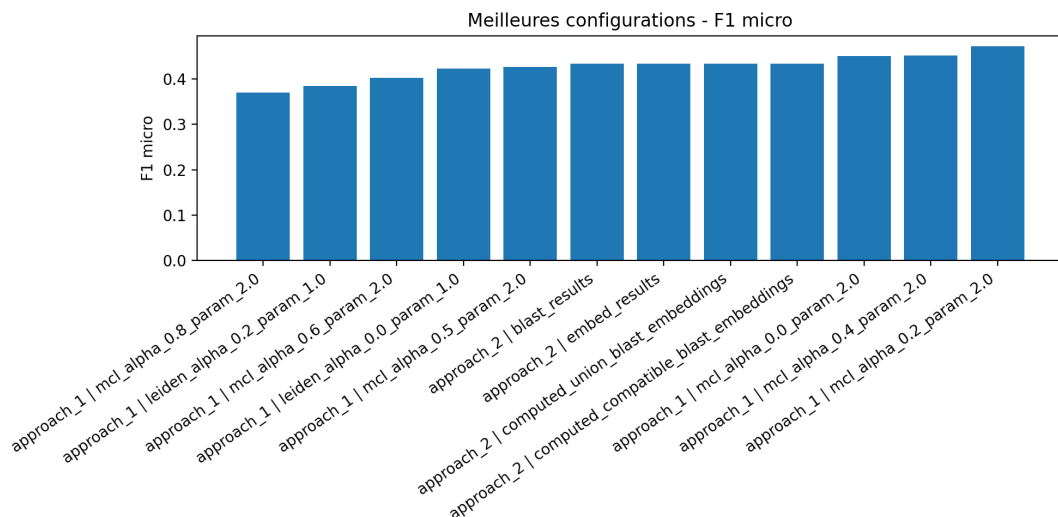


FIGURE 27 – Comparaison finale de la couverture, de la précision, du rappel et du F1-score pour les meilleures configurations.

La comparaison finale met en évidence un compromis entre couverture et précision. La deuxième approche obtient la meilleure couverture, car elle annote une plus grande proportion de protéines du jeu de test. Cependant, cette couverture plus élevée s'accompagne d'une baisse de précision, ce qui indique une propagation plus importante d'annotations incorrectes.

La première approche avec MCL et $\alpha = 0.2$ obtient le meilleur F1-score micro. Elle ne maximise pas la couverture, mais elle offre le meilleur compromis entre le nombre d'annotations retrouvées et la fiabilité des prédictions. Ce résultat montre que la fragmentation produite par MCL, qui pouvait apparaître comme une limite dans l'analyse structurale des clusters, devient ici un avantage pour la propagation : les clusters plus petits limitent les faux positifs.

Ces résultats montrent également que les embeddings apportent bien une information complémentaire à BLAST. En effet, la meilleure configuration n'est pas obtenue avec BLAST seul, mais avec une combinaison dans laquelle les embeddings ont un poids majoritaire. Les embeddings permettent donc de récupérer des relations fonctionnelles que BLAST seul ne suffit pas à exploiter.

Ainsi, les deux approches répondent à deux objectifs légèrement différents. La première approche, en particulier avec MCL et $\alpha = 0.2$, est la plus performante pour une prédiction fiable d'annotations. La deuxième approche est plus couvrante, mais moins précise, et peut donc être utilisée comme méthode exploratoire pour proposer davantage d'annotations candidates.

Conclusion L'analyse des annotations PFAM montre que l'ajout des embeddings améliore l'exploitation du graphe de similarité pour la propagation fonctionnelle. La fusion directe des poids dans la première approche permet d'obtenir le meilleur compromis global lorsque MCL est utilisé avec $\alpha = 0.2$. La deuxième approche confirme la complémentarité entre BLAST et embeddings, mais sa propagation alternée augmente le risque de faux positifs.

Notre conclusion est donc que les embeddings sont utiles pour enrichir l'information issue de BLAST, mais qu'ils doivent être intégrés de manière contrôlée. Dans nos expériences, la stratégie la plus fiable consiste à combiner les scores BLAST et embeddings avant clustering, puis à propager les annotations dans les clusters obtenus avec MCL.

16 Conclusion et Perspectives

16.1 Conclusion

Ce travail a permis de concevoir et de mettre en œuvre une méthodologie dédiée à l'analyse et à l'annotation fonctionnelle automatique des protéines, s'appuyant sur des approches de clustering.

Voici les quatre étapes mises en place :

1. **Préparation des données** : une phase amont essentielle incluant l'extraction des annotations PFAM, la génération embeddings à partir de ProteinBERT, ainsi qu'un découpage des jeux de données (*train* et *test*).
2. **Construction des graphes** : trois graphes ont été modélisés et comparés pour capturer au mieux les relations entre les protéines : un graphe **BLAST** basé sur la similarité des séquences, un graphe d'**embeddings** exploitant la similarité cosinus des embeddings, et un graphe **hybride** combinant ces deux graphes de manière pondérée.
3. **Étude du clustering** : une analyse comparative a été menée entre deux algorithmes de clustering : *Markov Clustering aLgorithm* (MCL) et *Leiden*. L'optimisation de leurs hyperparamètres respectifs (le paramètre d'inflation r pour MCL, la résolution γ pour Leiden, et le facteur de pondération α pour le graphe hybride) a été guidée par l'étude de la fragmentation des clusters, de la co-distance, de la coupe normalisée et des temps de calcul associés.
4. **Évaluation biologique et prédictive** : enfin, la pertinence du système a été validée à travers la mesure de la cohérence des clusters au regard des familles PFAM, complétée par une évaluation des performances de la propagation d'annotations à l'aide de métriques standards telles que la précision, le rappel, le F1-score et le taux de couverture.

16.2 Ouverture et Perspectives

Bien que la méthodologie développée offre des fondations solides pour l'annotation de séquences, plusieurs axes d'amélioration et d'ouverture scientifique peuvent être envisagés pour la suite des travaux :

- **Intégration de la structure 3D (AlphaFold)** : l'exploitation des repliements tridimensionnels modélisés par des outils tels qu'AlphaFold. L'apport des structures permettrait d'annoter efficacement des protéines distantes sur le plan de la séquence d'acides aminés (faible score BLAST) mais partageant une architecture tridimensionnelle similaire et, par conséquent, une fonction biologique commune.
- **Impact des propriétés dimensionnelles des séquences** : il conviendra d'analyser plus précisément l'influence de la longueur et des dimensions des séquences protéiques.
- **Gestion des incompatibilités fonctionnelles au niveau biologique** : un dernier axe d'étude consisterait à analyser l'incompatibilité des annotations au niveau biologique.

Références

- [1] Nadav Brandes, Dan Ofer, Yam Peleg, Nadav Rappoport, and Michal Linial. Protein-BERT : a universal deep-learning model of protein sequence and function. *Bioinformatics*, 38(8) :2102–2110, 02 2022.
- [2] Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C. Lawrence Zitnick, Jerry Ma, and Rob Fergus. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *PNAS*, 2019.
- [3] Stijn Marinus van Dongen. *Graph clustering by Flow Simulation*. PhD thesis, University of Utrecht, 2000.
- [4] Ludo Waltman Vincent A. Traag and Nees Jan Van Eck. From louvain to leiden : guaranteeing well-connected communities. *Scientific Reports*, 9(5233), 2019.